Fundamentos IA

Redes Neurais

Prof. Luiz Eduardo S. Oliveira, DINF, UFPR.
www,inf.ufpr.br/lesoliveira

Redes Neurais

Neste módulo

- Introdução a redes neurais (Perceptron, MLP)
- Redes Neurais Convolucionais (CNN)
  - Transfer-Learning
- Recurrent Neural Networks (RNN)

Redes Neurais

Um breve histórico - Início (1940s-1960s)

- **1943**: McCulloch & Pitts
  - Proposta do primeiro modelo matemático de um neurônio artificial (binary threshold unit)
- **1958**: Perceptron (Frank Rosenblatt)
  - Primeiro modelo prático para classificação binária (uma camada)
- **1969**: Limitações do Perceptron (Minsky & Papert)
  - Prova de que o Perceptron de uma camada não consegue resolver problemas não lineares
  - Início do "**AI Winter**"

Redes Neurais

Um breve histórico - AI Winter (1970s-1980s)

- **1974**: (Backpropagation) (Paul Werbos)
  - Alguns aspectos do algoritmo de treinamento não estavam bem resolvidos. Restrições de hardware
- **1982**: Hopfield Networks (**John Hopfield**)
  - Redes Recorrentes com memória associativa
- **1986**: Backpropagation revisitado (Rumelhart, **Hinton**, Willians)
  - Popularização do Backprop para treinar redes neurais com várias camadas (MLP).
  - IJCNN
- **1989**: Teorema Universal da Aproximação (Cybenko, Hornik)
  - Prova de que uma rede neural com uma camada pode aproximar qualquer função contínua.

Redes Neurais

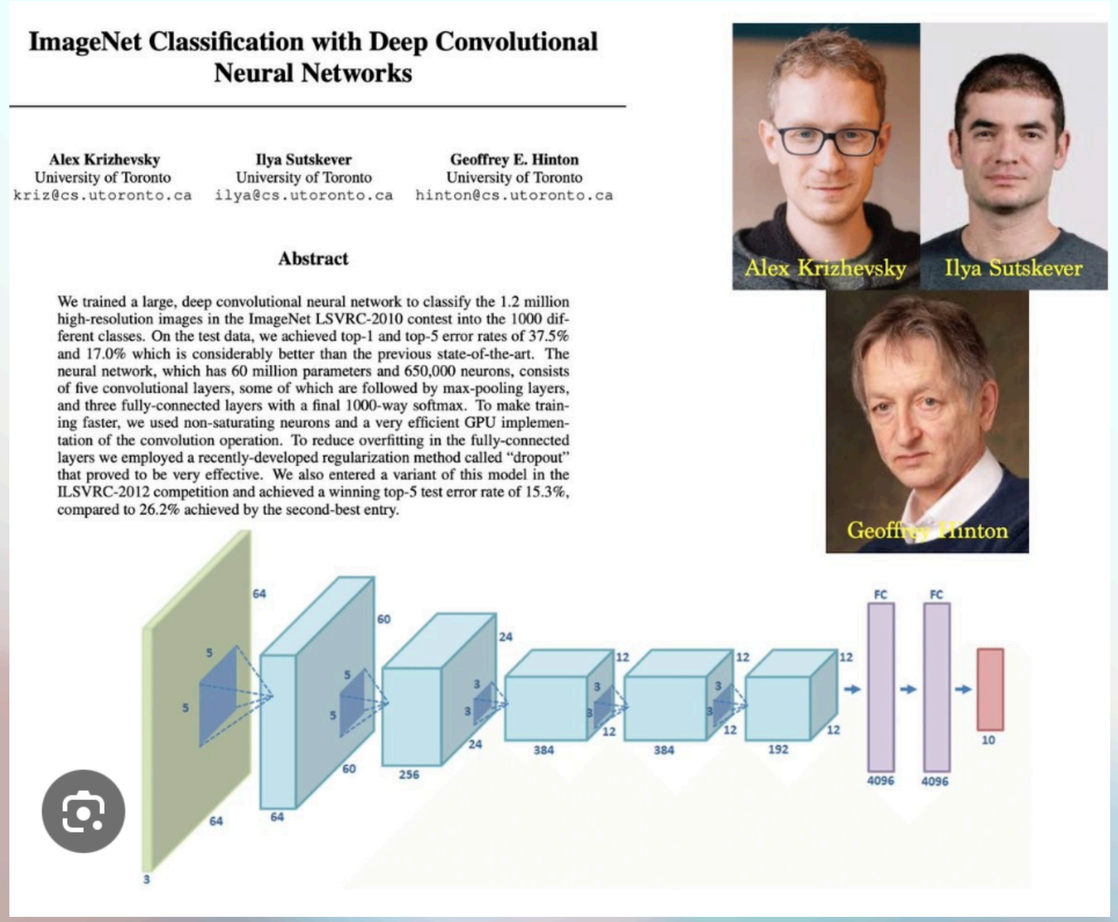
Um breve histórico - Rise & Fall (1990s-Começo 2000s)

- **1990s**: SVM superam as Redes Neurais
  - SVM (e métodos de Ensemble) superam redes neurais (limites computacionais e overfitting)
- **1997**: LSTM (Hochreiter & Schmidhuber)
  - LSTM mostra um avanço real para modelagem de sequências (voz, texto)
- Fim 1990s-2000s: **Segundo AI Winter**
  - Redes neurais são consideradas ineficientes para problemas em larga escala

Redes Neurais

Um breve histórico - Revolução Deep Learning (2010s-presente)

- **2012**: AlexNet (Krizhevsky, Sutskever, Hinton)
  - Rede neural profunda vence o ImageNet, baixando o erro de 26% para 15%



Redes Neurais

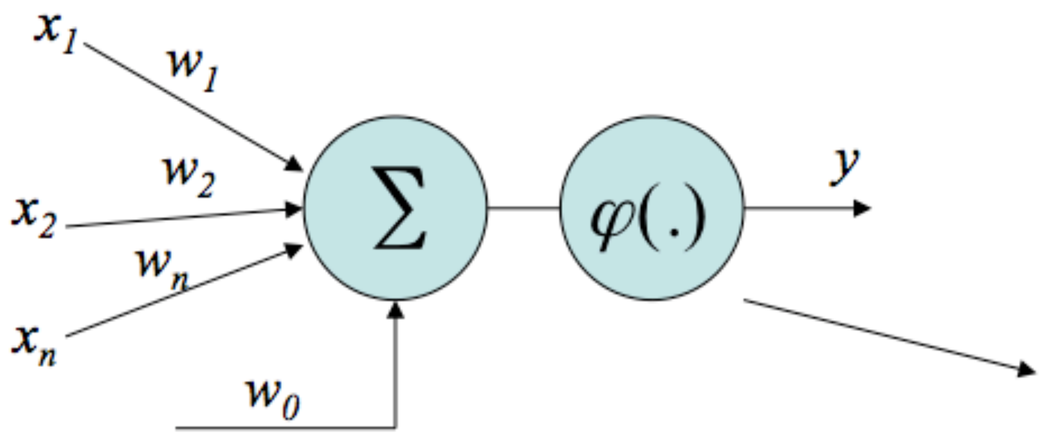
Um breve histórico - Revolução Deep Learning (2010s-presente)

- **2014**: Generative Adversarial Networks (GANs, Goodfellow)
  - Introduziu o conceito de treinamento adversário para modelos generativos
- **2017**: Transformers (Vaswani et al.)
  - Mecanismos de atenção que revolucionaram a área de NLP (BERT, GPT, etc..)
- **2020**: LLMs e IA multimodal
  - Modelos de linguagem, geração de imagens, video, etc...
  - Vision Transformers

Perceptron

Perceptron

Primeira e mais primitiva estrutra de rede neural



$$y = \varphi\left(\sum w_i \times x_i + w_0\right)$$

A função de ativação normalmente utilizada no perceptron é a hardlim (threshold)

$$f(x) = \begin{cases} 1 & x \geq 0 \\ 0 & x < 0 \end{cases}$$



Perceptron

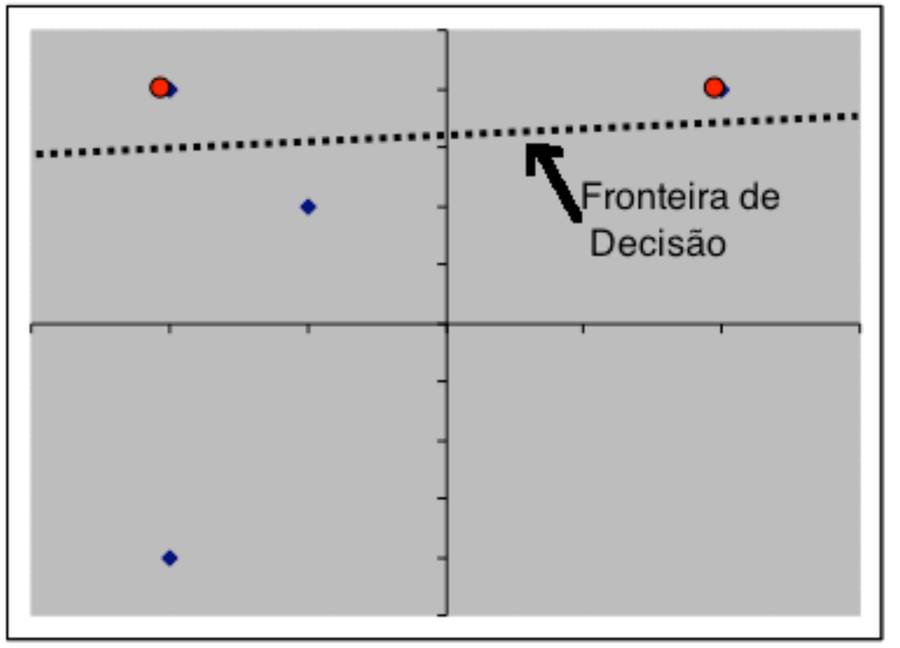
Algoritmo de treinamento

- Inicialização dos pesos (**w**) e bias (***b***)
- Enquanto o critério de parada não for satisfeito
  - Aplicar um padrão de entrada juntamente com seu rótulo t_j e verificar a saída ***y*** da rede.
  - Calcular o erro da saída $e = t_j - a$
  - Se $e \neq 0$
    - Atualizar pesos $w_i = w_i^{old} + e \times x_i$
    - Atualizar bias $b = b^{old} + e$

Perceptron

Considere o seguinte exemplo

Características (X)		Label (t)
2	2	0
-2	-2	1
-2	2	0
-1	1	1



Perceptron

- Considere b= 0 e w = (0,0)

Apresentando o primeiro padrão (x_1) a rede

- $y = \text{hardlim}([0,0][2,2]^t + 0) = \text{hardlim}(0) = 1$
- Erro: $e = t_j - y = 0 - 1 = -1$
- Como o erro $\neq 0$, atualizam-se os pesos e bias
  - $W = W^{old} + e \times x_i = [0,0] + (-1[2,2]) = [-2,-2]$
  - $b = b^{old} + e = 0 + (-1) = -1$

Perceptron

Apresentando o segundo padrão (x_2) a rede

- $y = \text{hardlim}([-2, -2][-2, -2]^t + (-1)) = \text{hardlim}(7) = 1$
- Erro: $e = t_j - y = 1 - 1 = 0$
- Como o erro $= 0$, pesos e bias não precisam ser atualizados

Apresentando o terceiro padrão (x_3) a rede

- $y = \text{hardlim}([-2, -2][-2, 2]^t + (-1)) = \text{hardlim}(-1) = 0$
- Erro: $e = t_j - y = 0 - 0 = 0$
- Como o erro $= 0$, pesos e bias não precisam ser atualizados

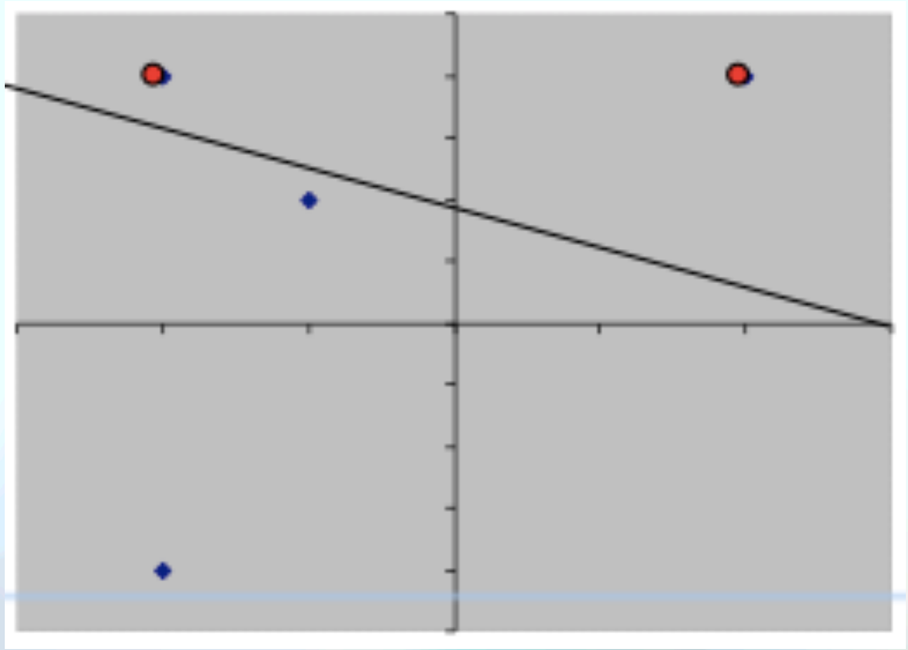
Perceptron

Apresentando o quarto padrão (x_4) a rede

- $y = \text{hardlim}([-2,-2][-1,1]^t + (-1)) = \text{hardlim}(-1) = 0$
- Erro: $e = t_j - y = 1 - 0 = 1$
- Como o erro $\neq 0$, atualizam-se os pesos e bias
  - $W = W^{old} + e \times x_i = [-2,-2] + (1[-1,1]) = [-3,-1]$
  - $b = b^{old} + e = -1 + 1 = 0$

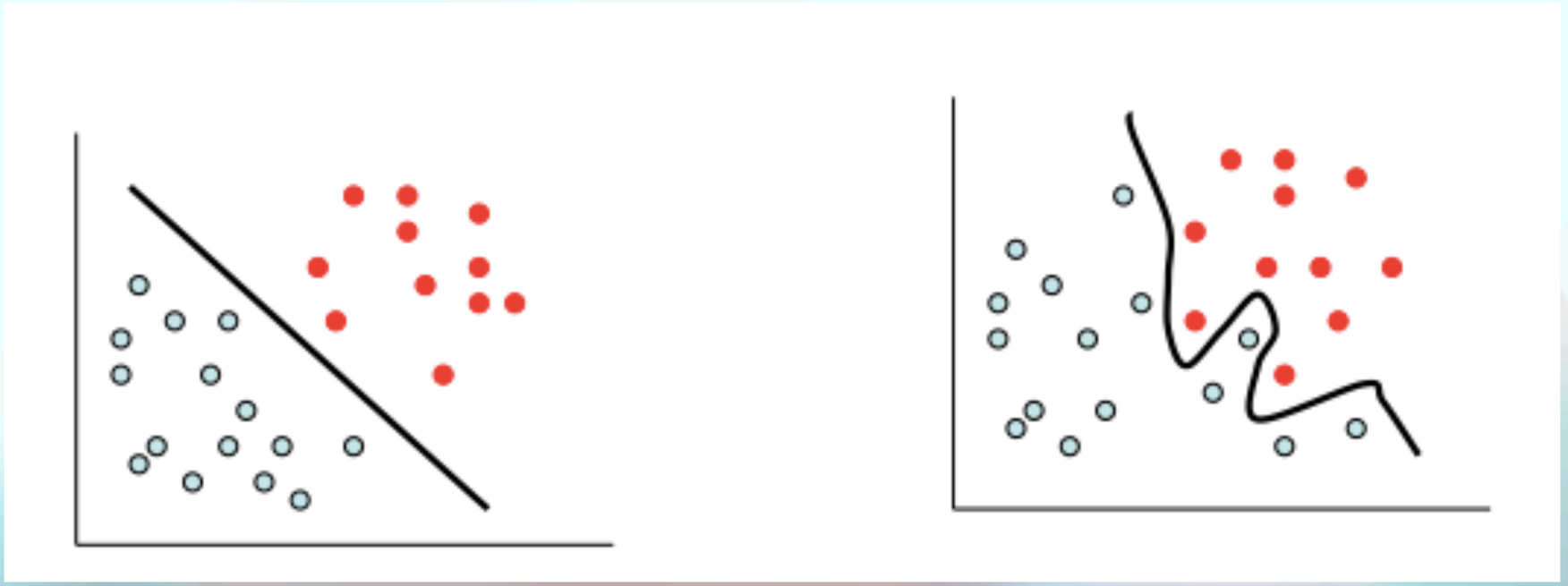
Perceptron

- Como todos os padrões foram apresentados a rede, o processo começa novamente, utilizando os pesos encontrados no último passo da primeira época (iteração).
- O algoritmo acaba quando o erro for zero (todos os padrões forem classificados corretamente) ou quando um certo número de épocas for alcançado.
- Para esse exemplo, os pesos finais são **w = [-1, -3] e b = 2**



Perceptron

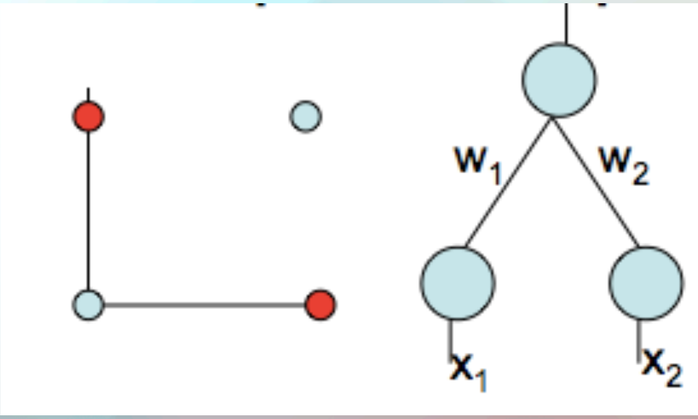
- Resolve problemas linearmente separáveis.
- Entretanto, nem sempre os problemas são linearmente separáveis.



Multi-Layer Perceptron

Problemas não Linearmente Separáveis

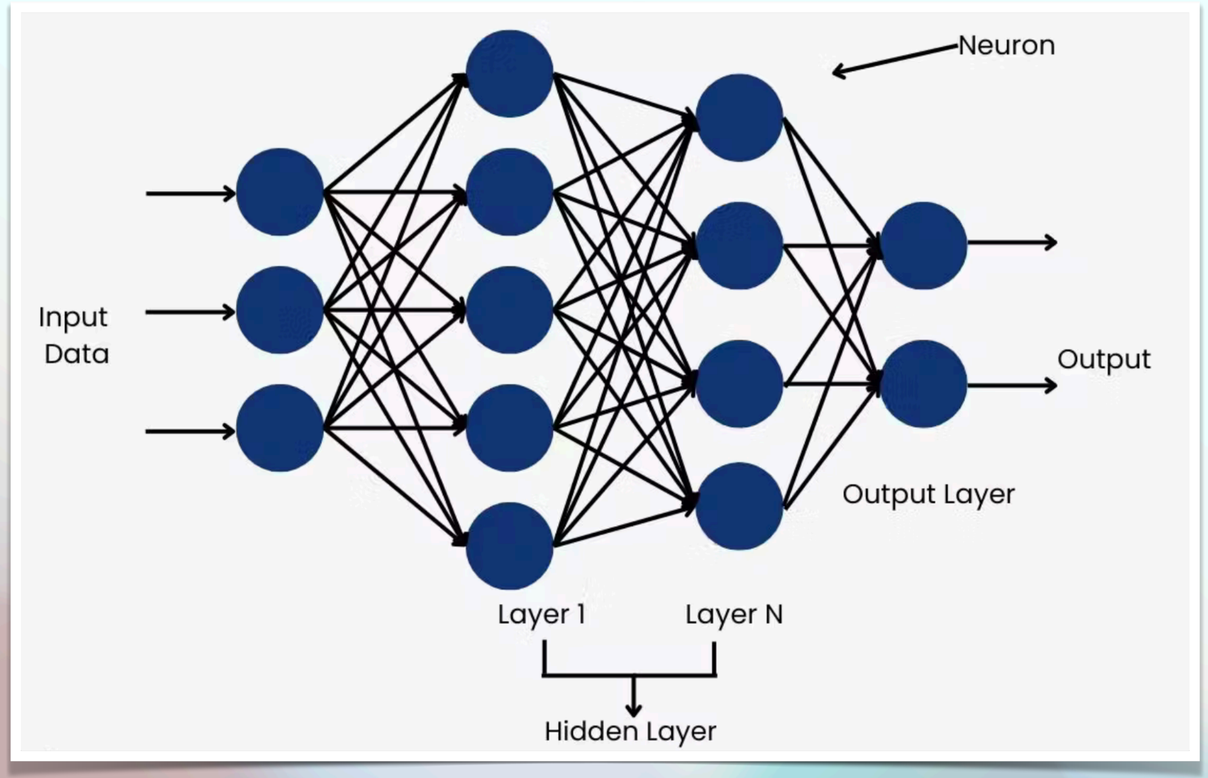
- Uma solução para resolver um problema não linearmente separável consiste em adicionar novas características ao vetor.
- Considere o problema XOR, o qual não pode ser resolvido com um perceptron.
- Nesse caso, a característica adicionada x_3 é a operação AND entre x_1 e x_2
- O fato de adicionarmos essa característica faz com que o problema torne-se linearmente separável



X1	X2	X3	Output
1	1	1	0
1	0	0	1
0	1	0	1
0	0	0	0

MLP - Multi-Layer Perceptron

- Outra maneira de resolver esse problema consiste em adicionar uma camada extra (camada escondida - hidden layer) entre as camadas de entrada e saída.
- Camadas escondidas podem ser interpretadas como extratores de características



MLP - Multi-Layer Perceptron

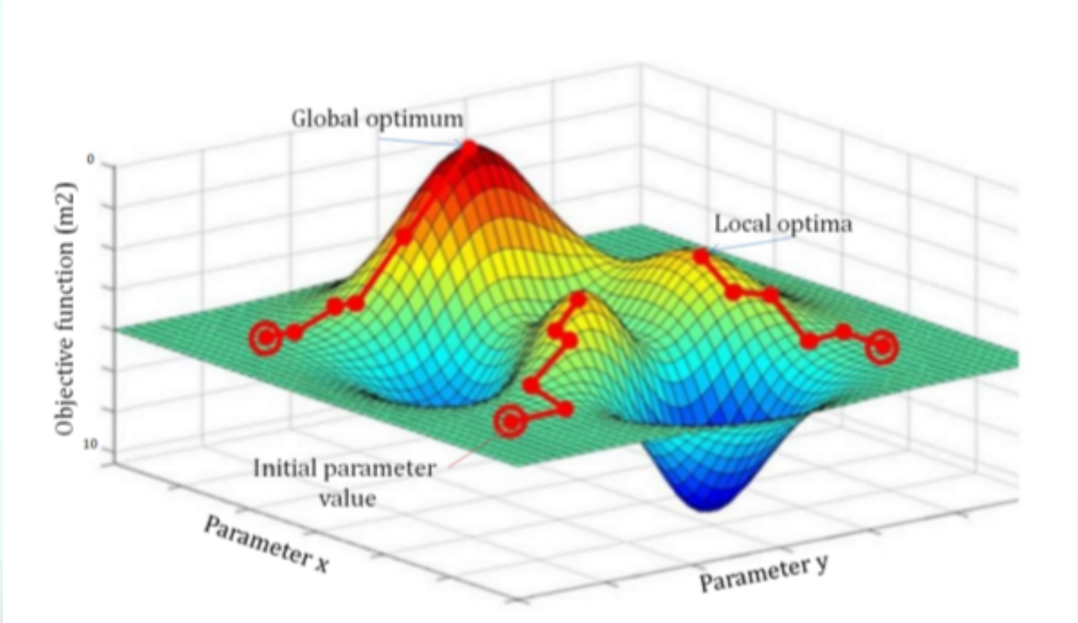
Problema de Atribuição de Créditos

- Quando temos uma camada escondida, surge o problema da atribuição de créditos ao neurônio desta camada.
  - Não existem "targets" como na camada de saída.
  - A solução foi o algoritmo conhecido como Backpropagation. (Werbos, Rumelhart)

MLP - Multi-Layer Perceptron

Backpropagation (Descida do Gradiente)

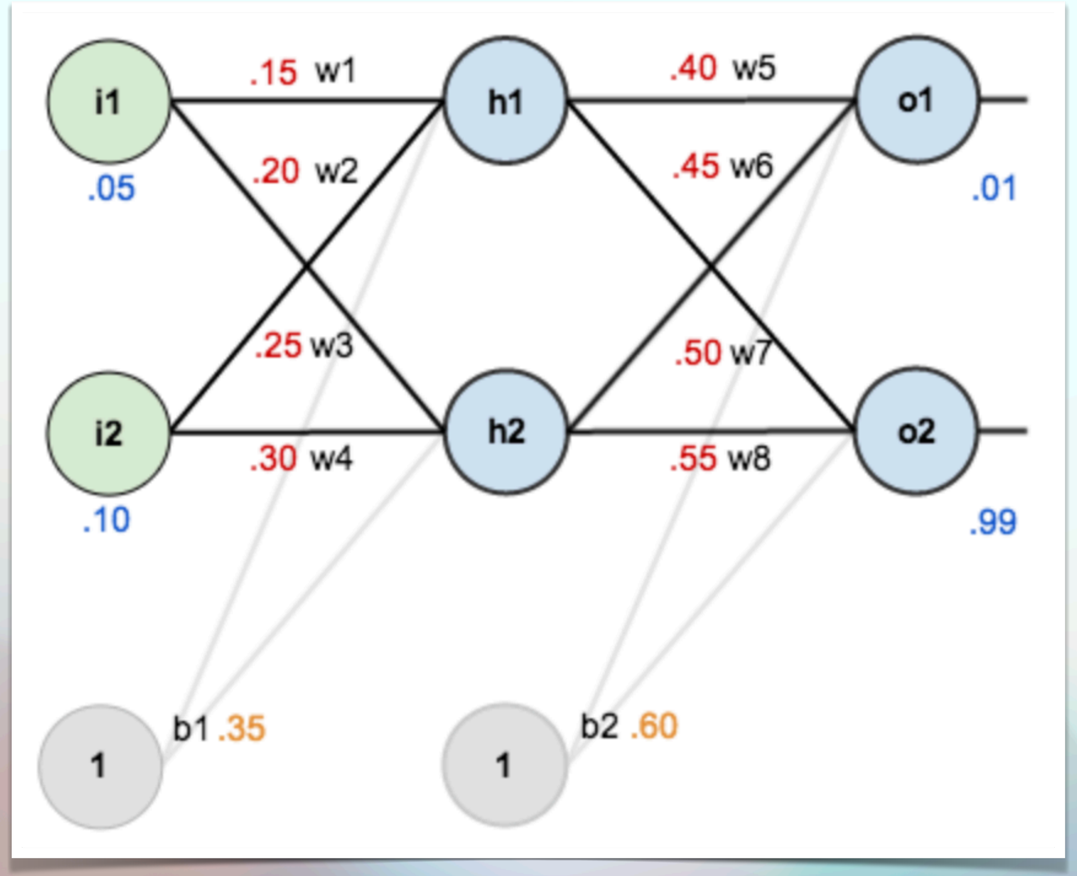
- Minimizar uma função de erro, a qual define o quanto a saída do modelo difere da saída desejada.
- Uma função bastante utilizada é a soma dos erros quadrados.
- BackProp encontra o conjunto de pesos que minimiza o erro através de uma técnica conhecida como a Descida do Gradiente (Gradient Descent)



BackPropagation

Exemplo

- Considere essa rede com duas entradas e duas saídas e os seguintes pesos associados
- Entradas: [0.05, 0.10]
- Saídas Esperadas: [0.01, 0.99]



BackPropagation - Forward

$net_{h1} = w_1 * i_1 + w_2 * i_2 + b_1 * 1$

$net_{h1} = 0.15 * 0.05 + 0.2 * 0.1 + 0.25 * 1 = 0.3775$

Em seguida, aplicamos a função logística para obter a saída de h_1

$out_{h1} = \frac{1}{1+e^{-net_{h1}}} = \frac{1}{1+e^{-0.3775}} = 0.593269$

Repetindo o processo para h_2, temos:

$out_{h2} = 0.596884$

Repetimos esse processo para os neurônios da camada de saída, usando a saída dos neurônios da camada escondida como entradas.

$net_{o1} = w_5 * out_{h1} + w_6 * out_{h2} + b_2 * 1$

$net_{o1} = .0.4 * 0.593269 + 0.45 * 0.596884 + 0.6 * 1 = 1.105905$

$out_{o1} = \frac{1}{1+e^{-net_{o1}}} = \frac{1}{1+e^{-1.105905}} = 0.751365$

Repetindo o processo para h_2, temos:

$out_{o2} = 0.772928$

Note que as saídas são bem diferentes das saídas esperadas [0.01 e 0.99].

Próximo passo consiste em calcular o erro total.

BackPropagation - Cálculo do Erro

$E_{total} = \sum \frac{1}{2}(target - output)^2$

O $\frac{1}{2}$ é incluído para que o expoente seja cancelado quando diferenciarmos mais adiante. O resultado acaba sendo multiplicado por uma taxa de aprendizado de qualquer forma, então não há problema em introduzir essa constante aqui.

Por exemplo, a saída alvo para o_1 é 0.01 mas a saída da rede neural é 0.751365; portanto, o erro correspondente é

$E_{o1} = \sum \frac{1}{2}(target_{o1} - output_{o1})^2 = \frac{1}{2}(0.01 - 0.751365)^2 = 0.274811$

Repetindo esse processo para o_2 (lembrando que o target $= 0.99$), temos:

$E_o2 = 0.023560$

O erro total para a rede é a soma dos erros

$E_{total} = E_{o1} + E_{o2} = 0.274822 + 0.023560 = 0.298371$

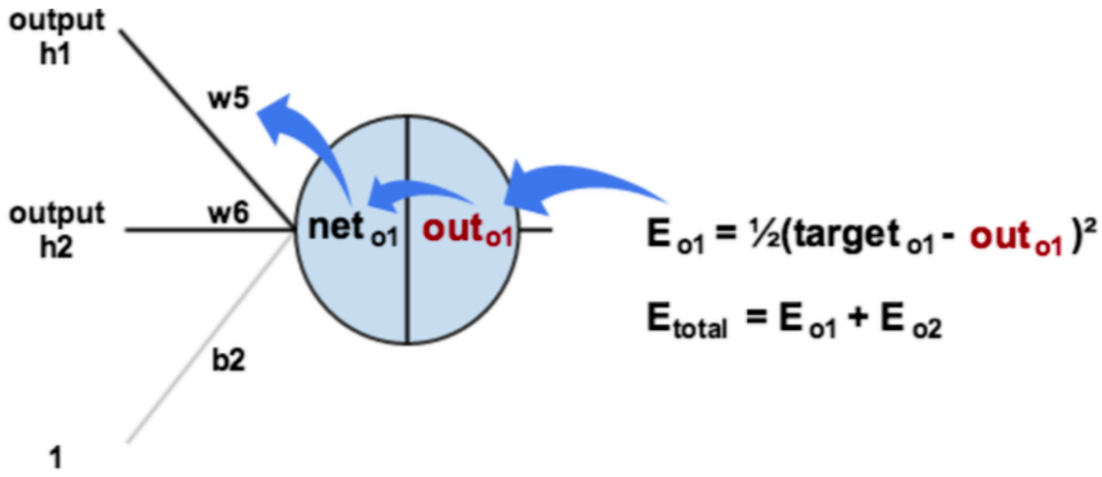
BackPropagation - Backward Pass

Considere w_5. Queremos saber quanto a mudança em w_5 afeta o erro total, também conhecido como $\frac{\partial E_{total}}{\partial w_5}$, ou seja, a "derivada parcial do E_{total} com relação a w_5". Ou ainda o gradiente em relação a w_5.

Aplicando a regra da cadeia (*chain rule*), sabe-se que

$$\frac{\partial net_{o1}}{\partial w_5} \times \frac{\partial out_{o1}}{\partial net_{o1}} \times \frac{\partial E_{total}}{\partial out_{o1}} = \frac{\partial E_{total}}{\partial w_5}$$

Visualmente, isso é oque acontece

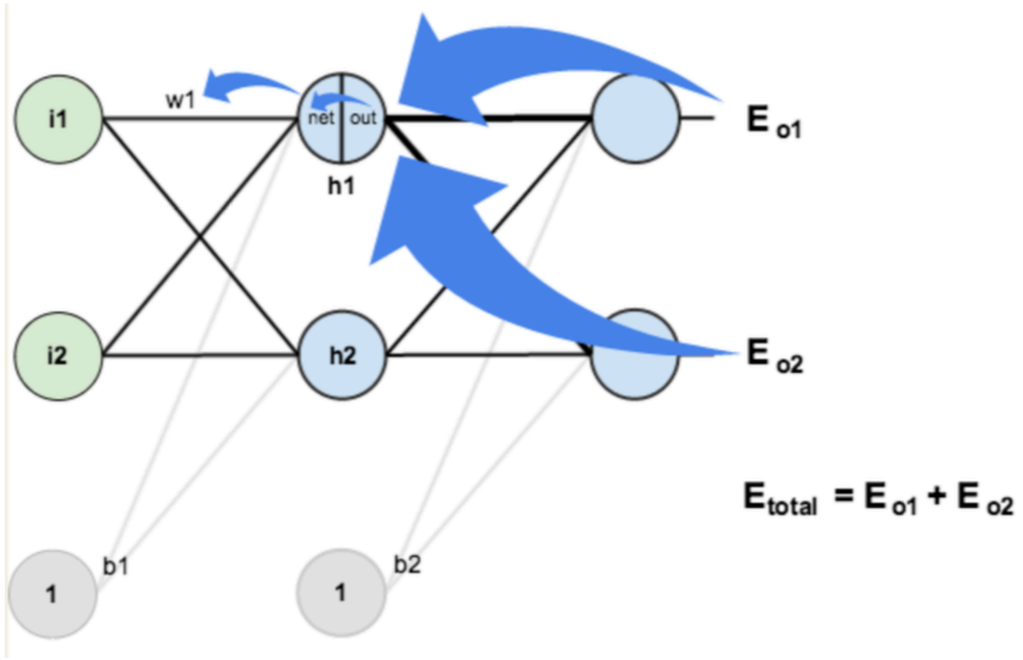


BackPropagation - Backward Pass

Na sequência, continuamos com a retropropagação calculando novos valores para w_1, w_2, w_3 e w_4.

Para isso precisamos calcular a derivada parcial do E_{total} com relação a w_1, o qual é dado por

$$\frac{\partial E_{total}}{\partial w_1} = \frac{\partial E_{total}}{\partial out_{h1}} \times \frac{\partial out_{h1}}{\partial net_{h1}} \times \frac{\partial net_{h1}}{\partial w_1}$$

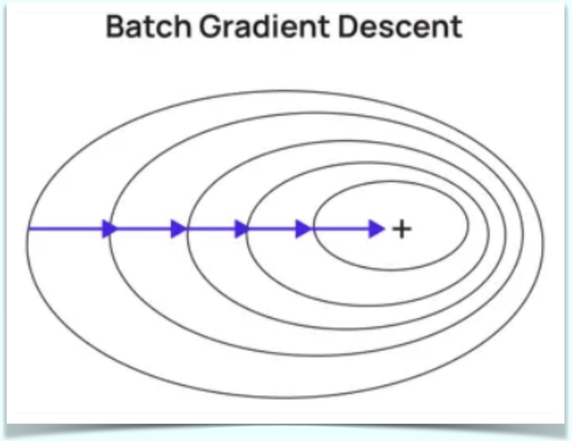


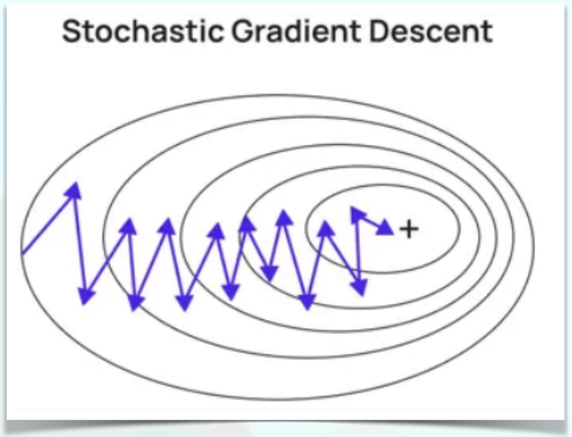
Exemplo completo do back-propagation está disponível no Moodle.

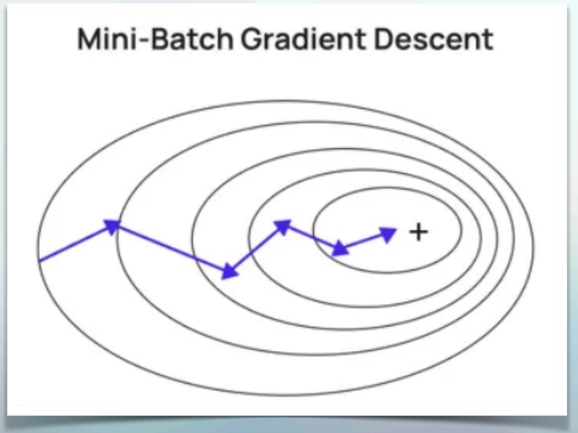
MLP - Multi-Layer Perceptron

Backpropagation (Descida do Gradiente)

- Batch
  - Usa todo o conjunto de dados a cada iteração.
  - Calcula o erro para cada exemplo e faz a média para definir a direção do gradiente.
- Estocástico
  - Para cada exemplo de treinamento, o algoritmo calcula a direção do Gradiente
- MiniBatch
  - Combina os dois anteriores (tamanho típico de minibatch: 32, 64, 128)
- ADAM (Adaptive Moment Estimation)
  - Técnica mais utilizada atualmente.







MLP - Multi-Layer Perceptron

Aspectos Práticos

- Alguns pontos devem ser considerados na utilização de redes neurais MLP
  - Leitura recomendada “Efficient Backprop”, Y. LeCun et al, 1998
    - Taxa de aprendizagem (velocidade de convergência)
    - Momentum (evitar mínimos locais)
    - Shuffle (evitar catastrophic forgetting)
    - Normalização (características com média próxima a zero) $Z = \frac{X - \mu}{\sigma}$

MLP - Multi-Layer Perceptron

Exemplo MNIST

- Base de dados criada nos anos 90 para avaliação de algoritmos de aprendizagem de máquina
- 10 classes de dígitos manuscritos
- Estrutura da rede

```
Input (784)
  ↓
[512-Dense → BatchNorm → Dropout]
  ↓
[256-Dense → BatchNorm → Dropout]
  ↓
10-Softmax
```



Exemplo MLP usando Keras

Base de dados

```
# Load and preprocess MNIST data
(X_train, y_train), (X_test, y_test) = mnist.load_data()
X_train = X_train.reshape(-1, 784).astype('float32') / 255.0  # Flatten and normalize
X_test = X_test.reshape(-1, 784).astype('float32') / 255.0
y_train = to_categorical(y_train, 10)  # One-hot encoding
y_test = to_categorical(y_test, 10)
```

Estrutura da rede

```
model = Sequential([
    Dense(512, activation='relu', input_shape=(784,)),
    BatchNormalization(),
    Dropout(0.2),
    Dense(256, activation='relu'),
    BatchNormalization(),
    Dropout(0.2),
    Dense(10, activation='softmax')
])

# Compile with Adam optimizer
model.compile(
    optimizer=Adam(learning_rate=0.001),
    loss='categorical_crossentropy',
    metrics=['accuracy']
)

# Train the model and store history
history = model.fit(
    X_train, y_train,
    batch_size=128,
    epochs=20,
    validation_data=(X_test, y_test)
)
```

Características são os pixels das imagem

```
Input (784)
  ↓
[512-Dense → BatchNorm → Dropout]
  ↓
[256-Dense → BatchNorm → Dropout]
  ↓
10-Softmax
```

Google Colab

Exemplo usando Keras

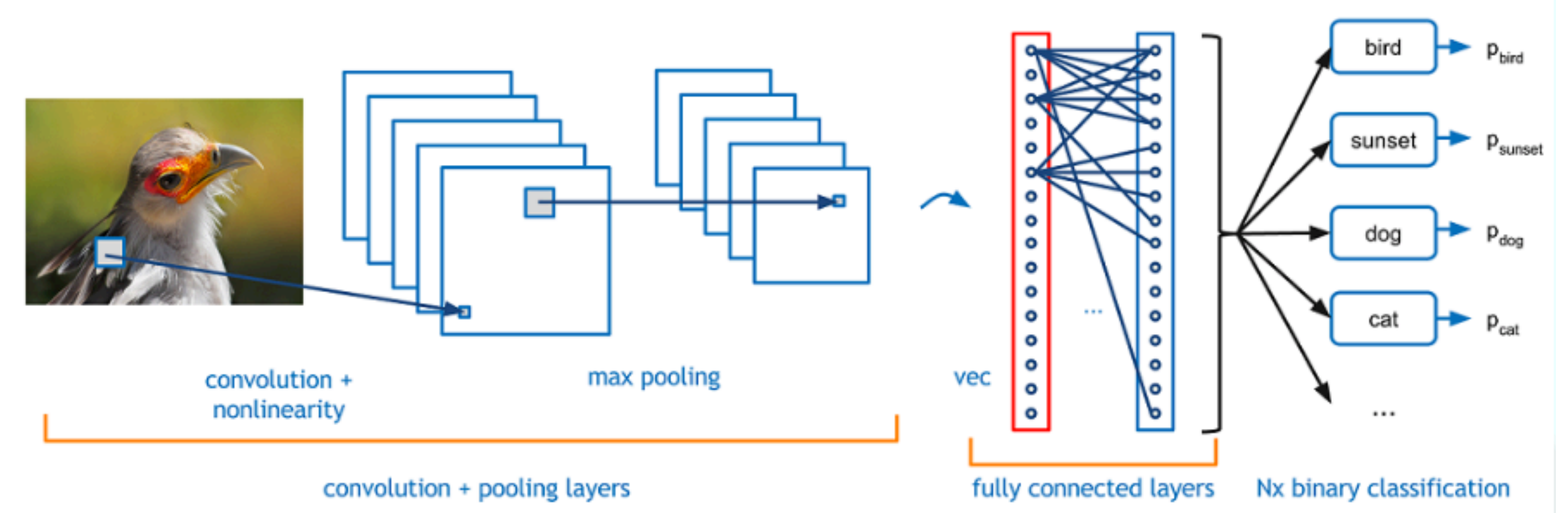


Convolutional Neural Networks

Aprendendo a Representação

- No exemplo anterior usamos os valores dos pixels como características, a qual foi definida empiricamente por um especialista humano.
- Note que o desempenho de qualquer método de aprendizagem de máquina depende da escolha da representação (características)
- O ideal seria tornar os métodos de aprendizagem de máquina menos dependente do especialista humano.
- Aprender representações discriminantes a partir dos dados (**Representation Learning**).
  - **Convolutional Neural Network (CNN)**

Redes Neurais Convolucionais (CNN)



- Uma CNN é composta de quatro tipos básicos de camadas
  - Convolução
  - Camada de não linearidade (RELU)
  - Pooling (Agregação)
  - Classificação (Fully Connected)

CNN

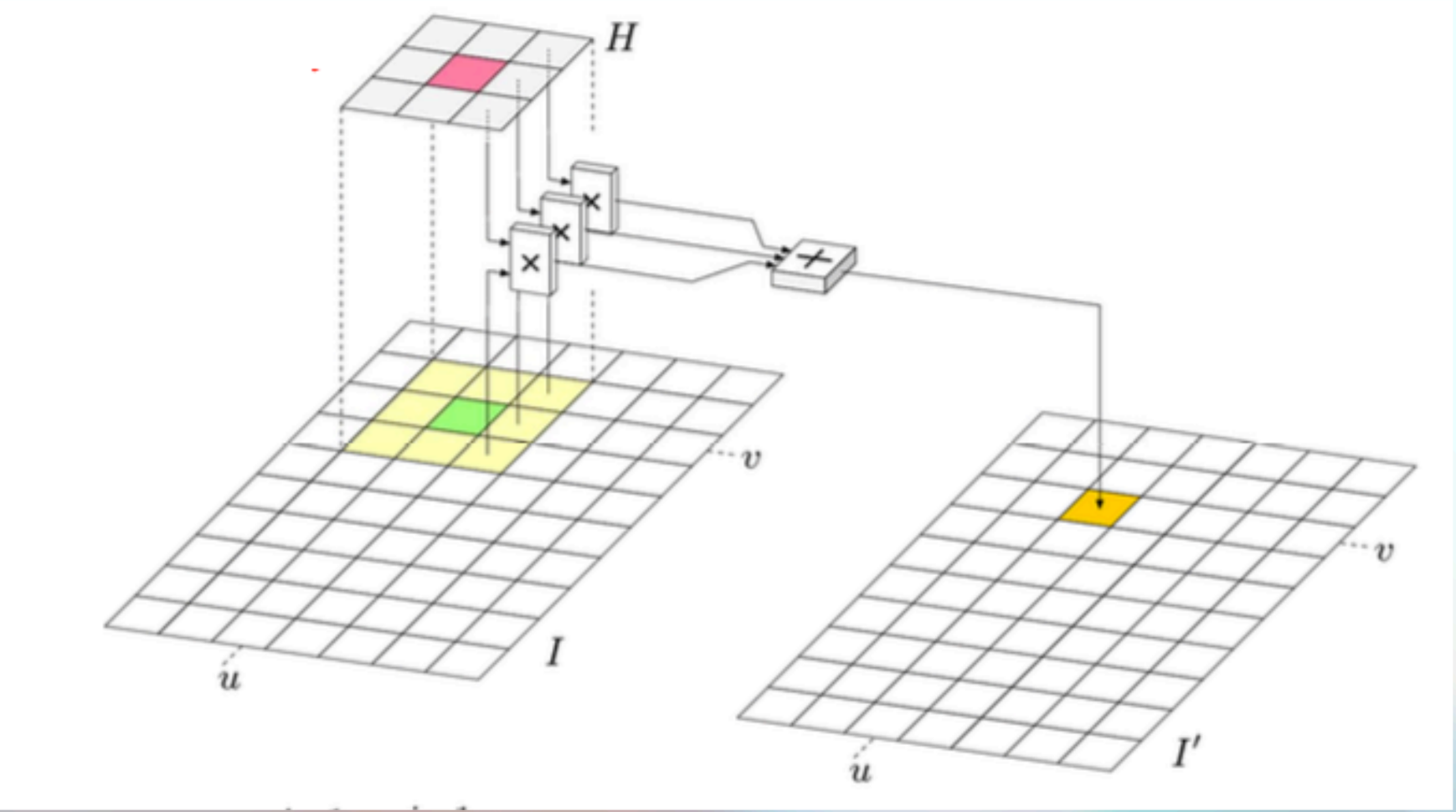
Convolução

- Imagine uma pequena lanterna que você usa em uma grande imagem para detectar algum padrão (borda, textura ou forma)
  - A pequena área iluminada é um filtro (chamado de kernel)
  - O processo de deslizar esse filtro através de toda a imagem é a convolução
  - Para uma imagem 2D e um filtro H, a convolução na posição (i,j) é dada por

$$I'(u,v) = \sum_m \sum_n I(u+i, v+i) \cdot H(i,j)$$

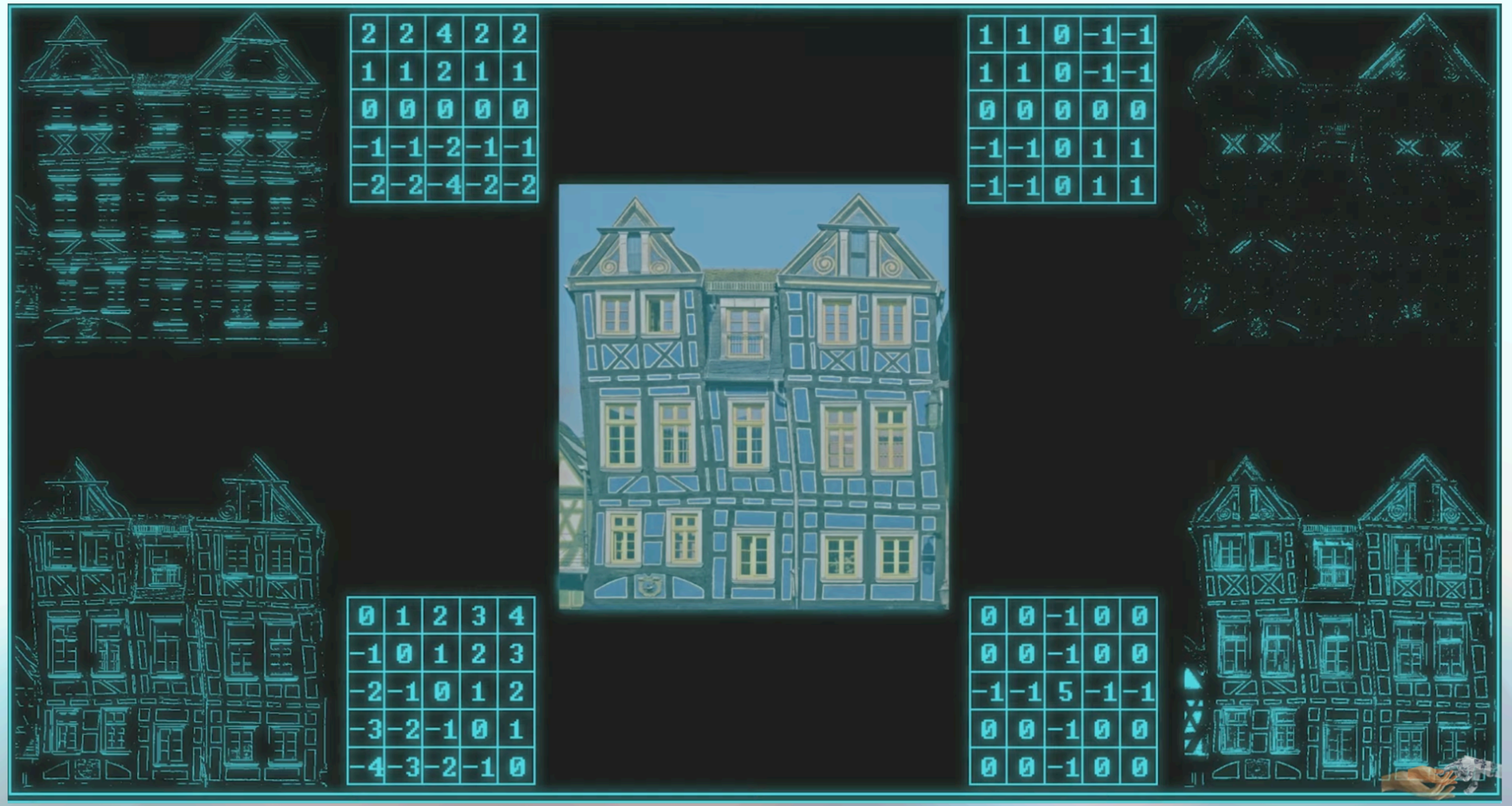
CNN

Convolução



CNN

Convolução



CNN

Exemplos de filtros aprendidos

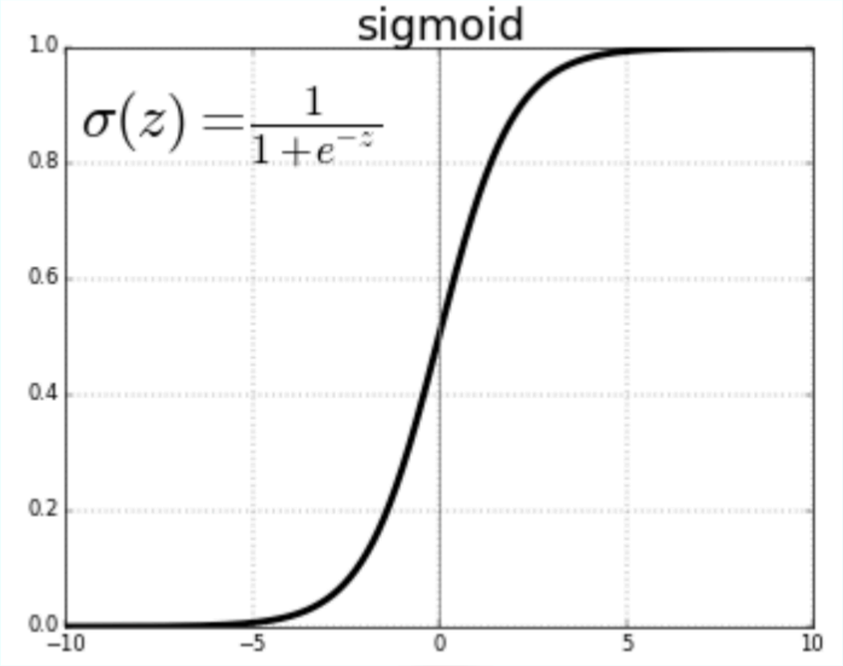
- Na prática, a CNN aprende o valor desse filtros durante o treinamento da rede.
- Precisamos especificar, porém, parâmetros como o número de filtros, tamanho, arquitetura da rede, etc.

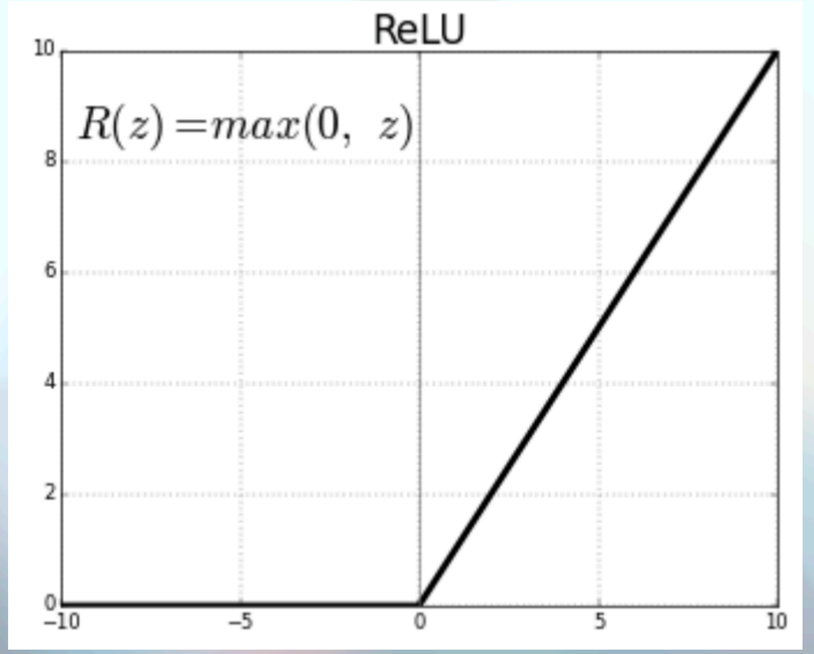


CNN

Camada de Ativação (ReLU)

- Operação aplicada no pixel (element wise) que basicamente substitui todos os valores negativos do Feature Map por zero.
- O objetivo da ReLU é introduzir não linearidade na rede, uma vez que a maioria dos dados no mundo real é não linear (e a operação de convolução é linear).
  - Se a rede tiver somente operadores lineares (convolução), no fim temos uma função linear.
- Além disso, a ReLU resolve o problema do desaparecimento do gradiente (Vanish Gradient)
  - Valores maiores não são limitados a 1 como na sigmoid.

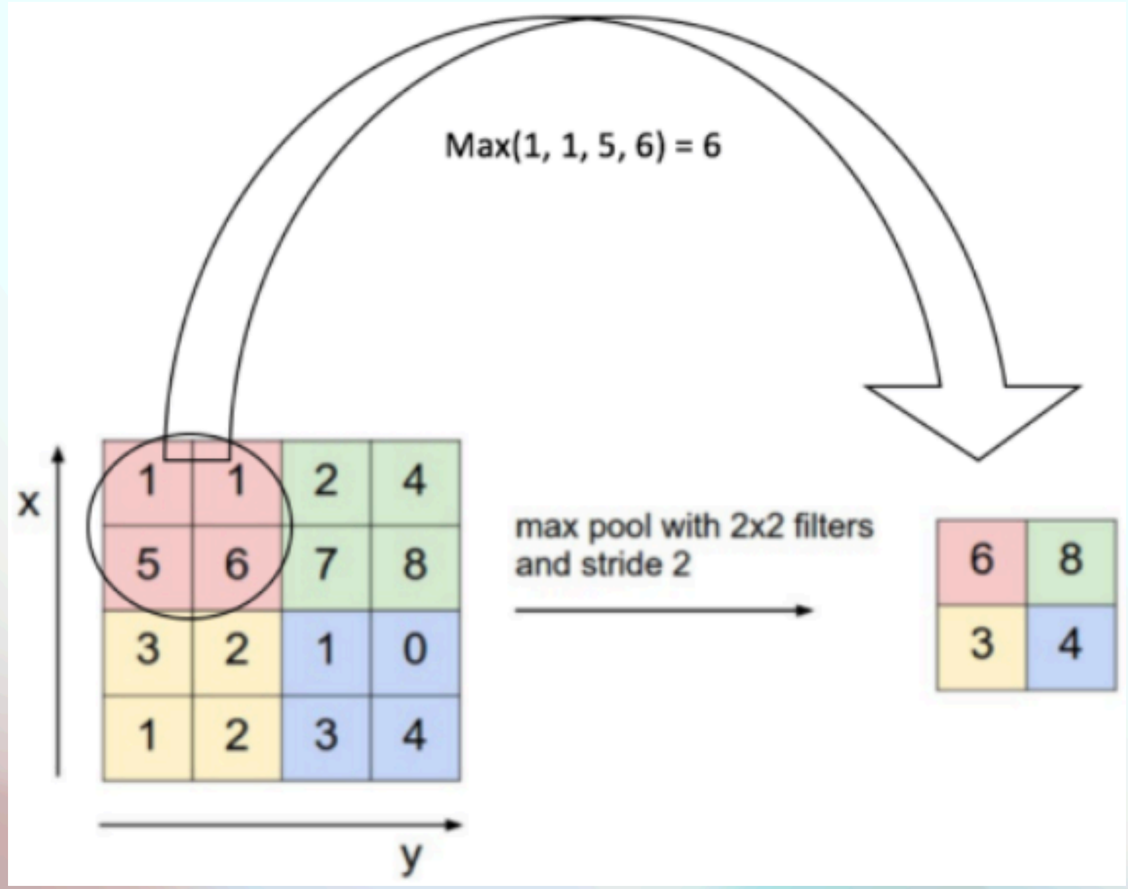




CNN

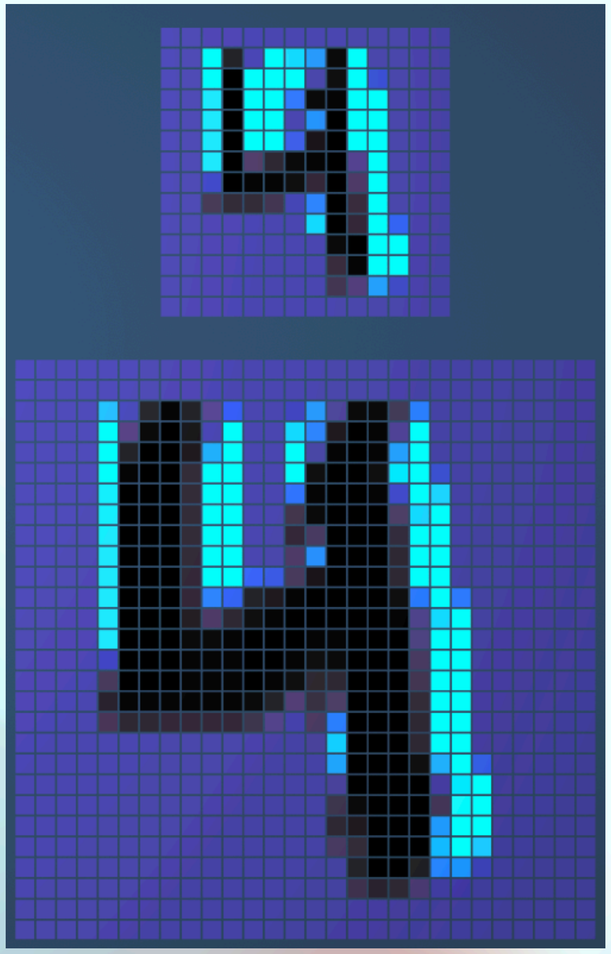
Pooling (Agregação)

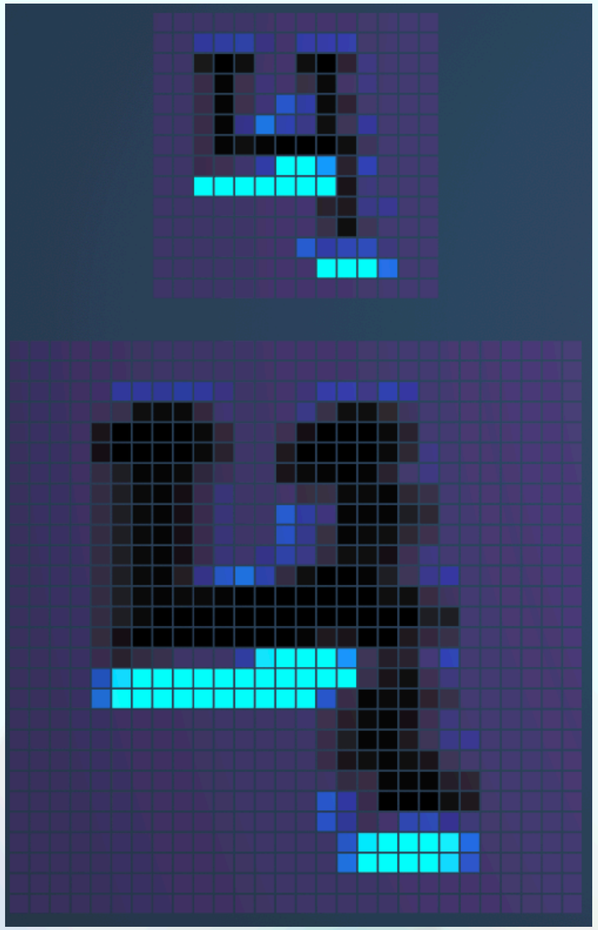
- Reduz a dimensionalidade de cada Feature Map tentando conservar as informações mais relevantes.
- Pode ser implementado de diversas maneiras, como por exemplo, Max, Média, Soma, etc.



CNN

Pooling (Agregação)





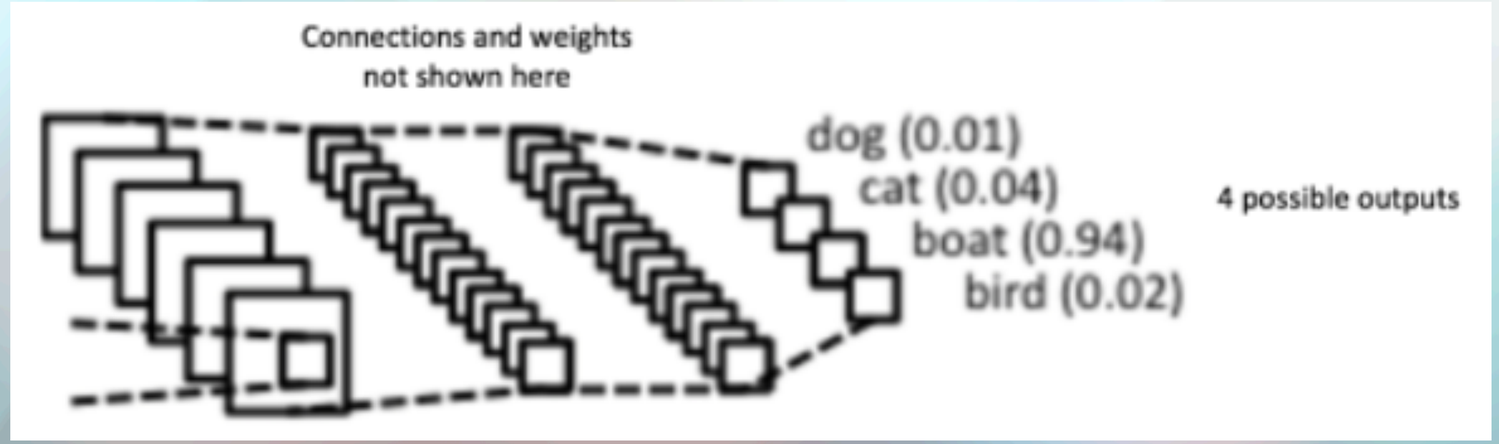
Reduz o feature map mantendo informações relevantes

Note a similaridade entre as diferentes resoluções

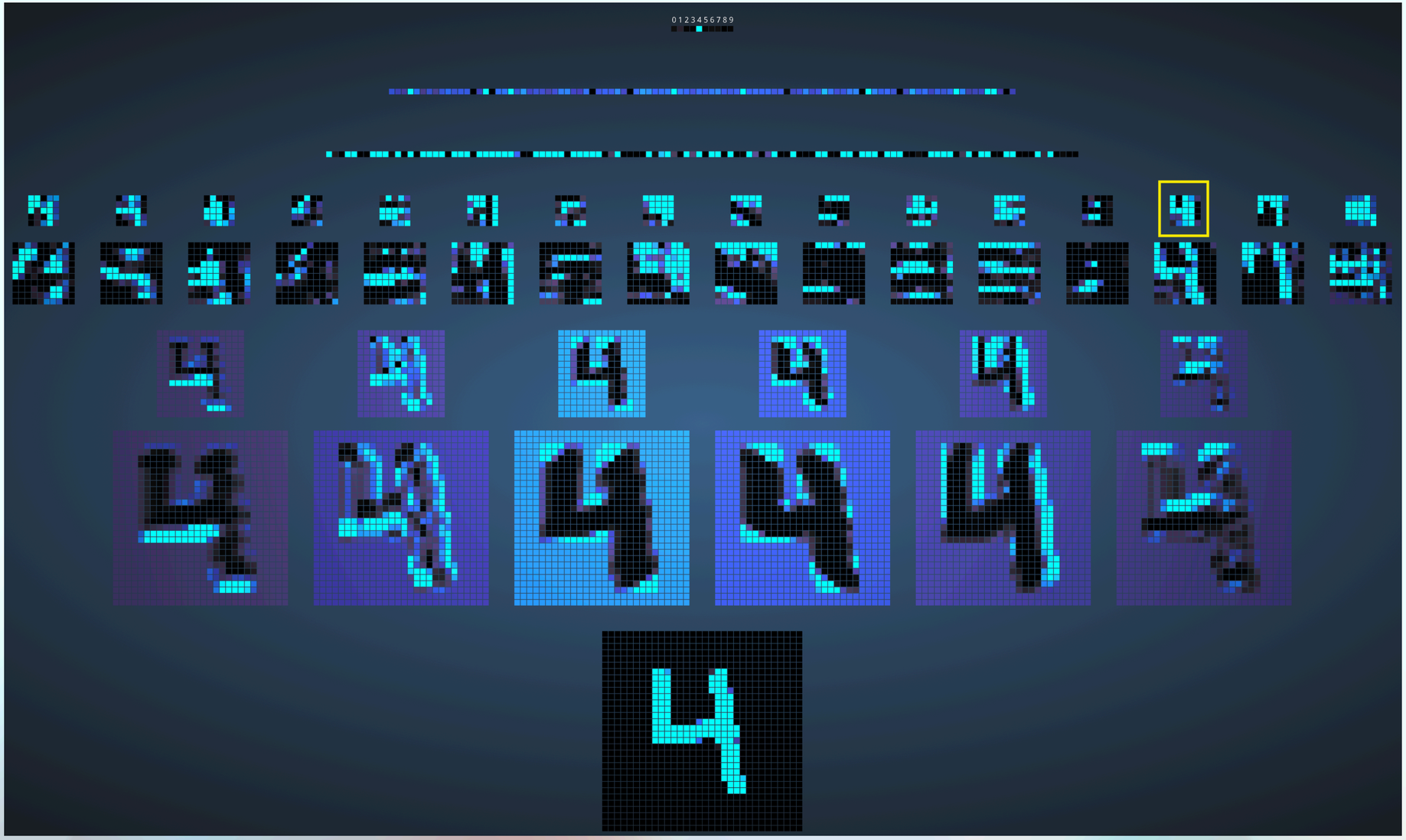
CNN

Classificação (Fully Connected Layer)

- Essa camada é uma MLP com função de ativação **softmax** na camada de saída (que garante estimação de probabilidades a posteriori), e também que a soma das saídas seja = 1
- A saída das camadas de convolução+ReLU+Pooling representam características de alto nível que servem como entrada da MLP.
- Essa representação pode ser utilizada com qualquer outro classificador.



CNN



Note que quanto mais profunda a rede, maior o nível de abstração.

Em alguns casos, podemos identificar padrões das camadas anteriores

CNN Visualization

Exemplo CNN usando Keras

```
# Build the CNN model
model = Sequential([
    # Convolutional Layer 1: 32 filters, 3x3 kernel, ReLU activation
    Conv2D(32, (3, 3), activation='relu', input_shape=(28, 28, 1)),
    # Max Pooling Layer 1: 2x2 pool size
    MaxPooling2D((2, 2)),

    # Convolutional Layer 2: 64 filters, 3x3 kernel, ReLU activation
    Conv2D(64, (3, 3), activation='relu'),
    # Max Pooling Layer 2: 2x2 pool size
    MaxPooling2D((2, 2)),

    # Flatten the output for dense layers
    Flatten(),

    # Fully Connected Layer: 128 neurons, ReLU activation
    Dense(128, activation='relu'),

    # Output Layer: 10 neurons (for 10 digits), softmax activation
    Dense(10, activation='softmax')
])

# Compile the model
model.compile(optimizer='adam',
              loss='categorical_crossentropy',
              metrics=['accuracy'])

# Train the model
history = model.fit(X_train, y_train,
                    epochs=5,
                    batch_size=64,
                    validation_split=0.2)
```

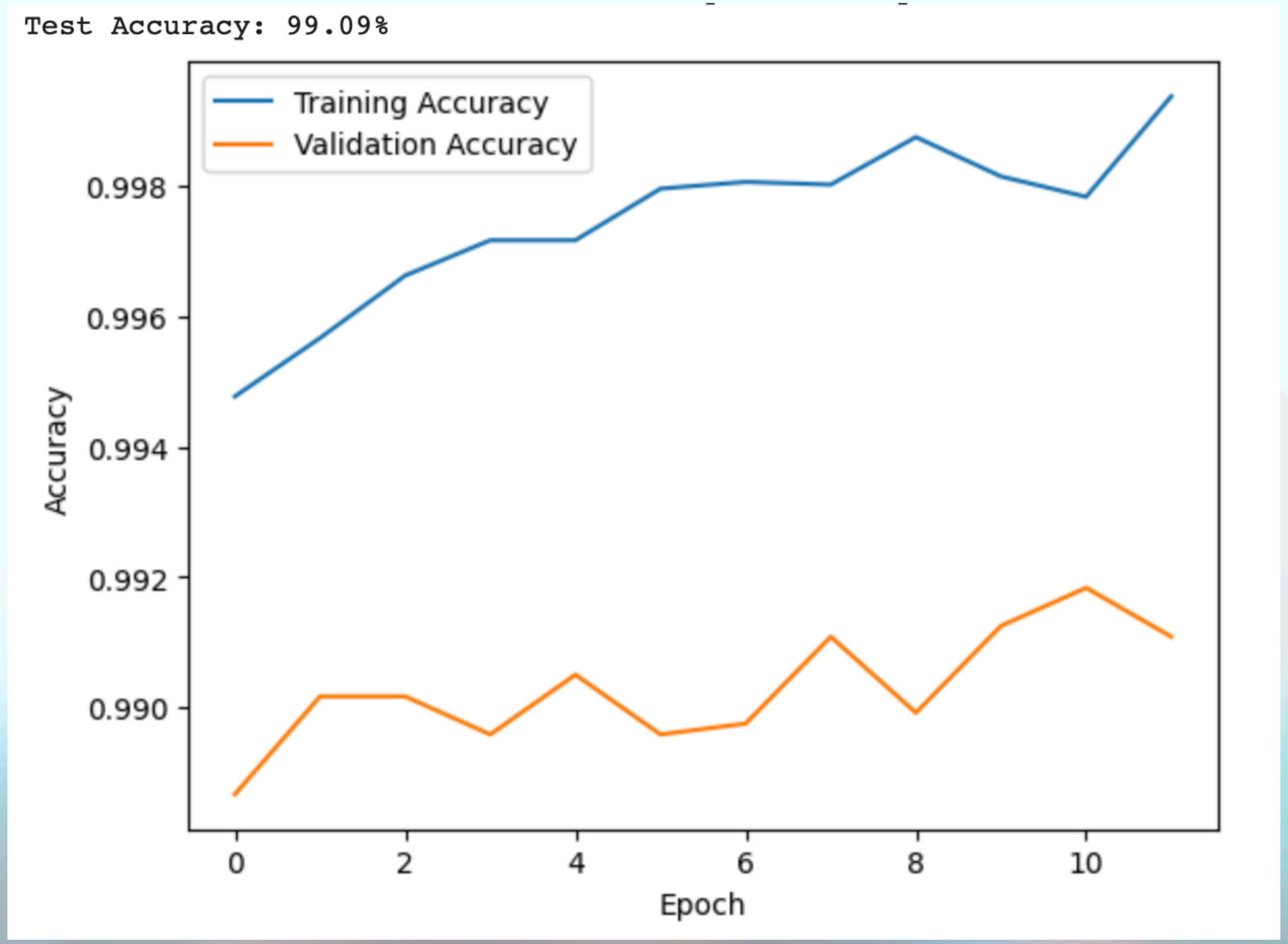
32 filtros

64 filtros

Google Colab

Exemplo CNN usando Keras

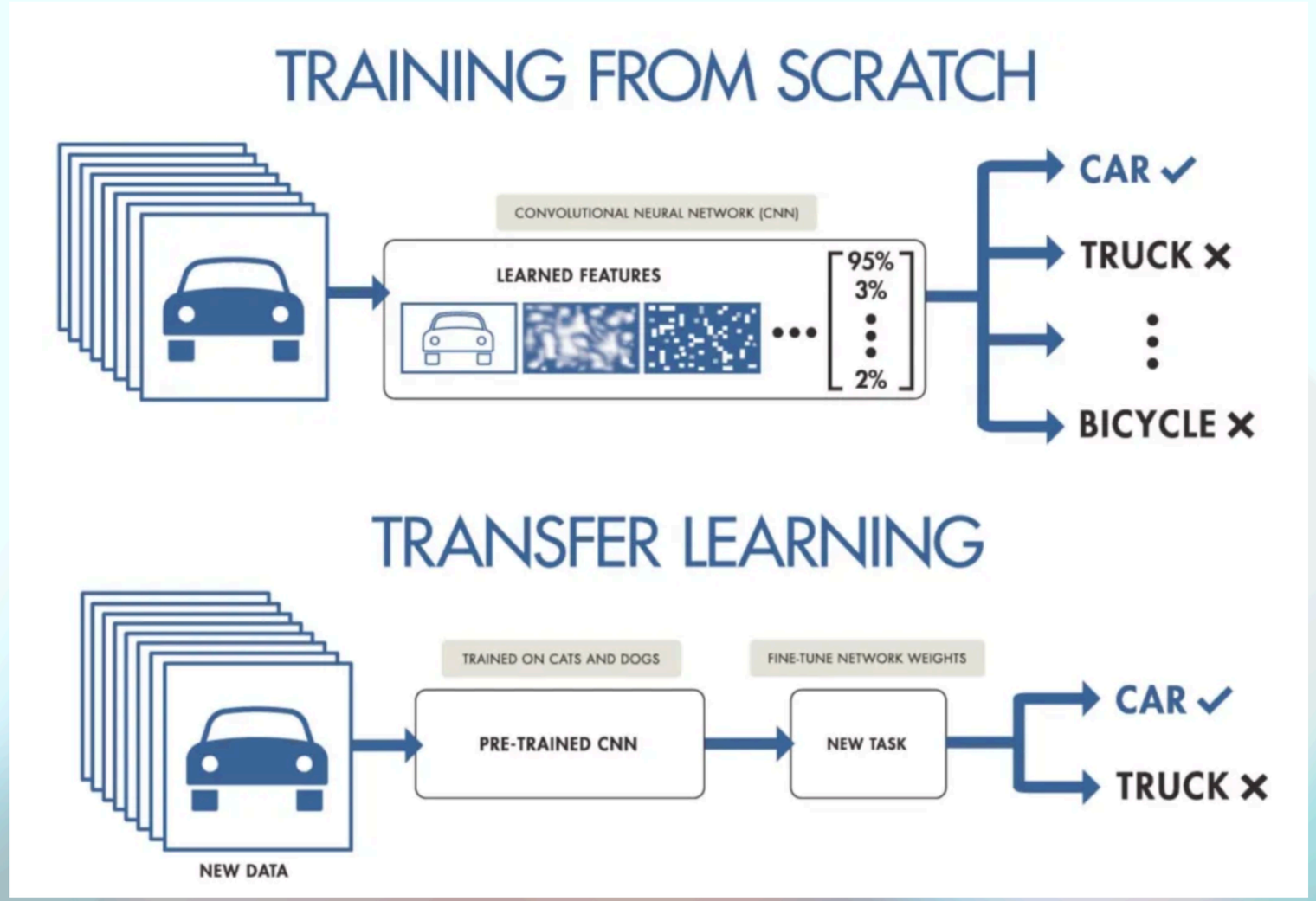
```
Test Accuracy: 99.09%
```



Transfer Learning

- O custo para treinar uma rede neural é alto, principalmente em grandes base de dados
- Uma solução é usar uma rede previamente treinada e usa-la na sua tarefa
  - Os modelos pré-treinados conseguem detectar muito bem características como bordas, formas e texturas.
- Alguns modelos populares
  - MobileNet, EfficientNet (dispositivos móveis)
  - ResNet50, VGG, DenseNet (Alta acurácia)
  - YOLO (detecção de objetos)

Transfer Learning



Transfer Learning

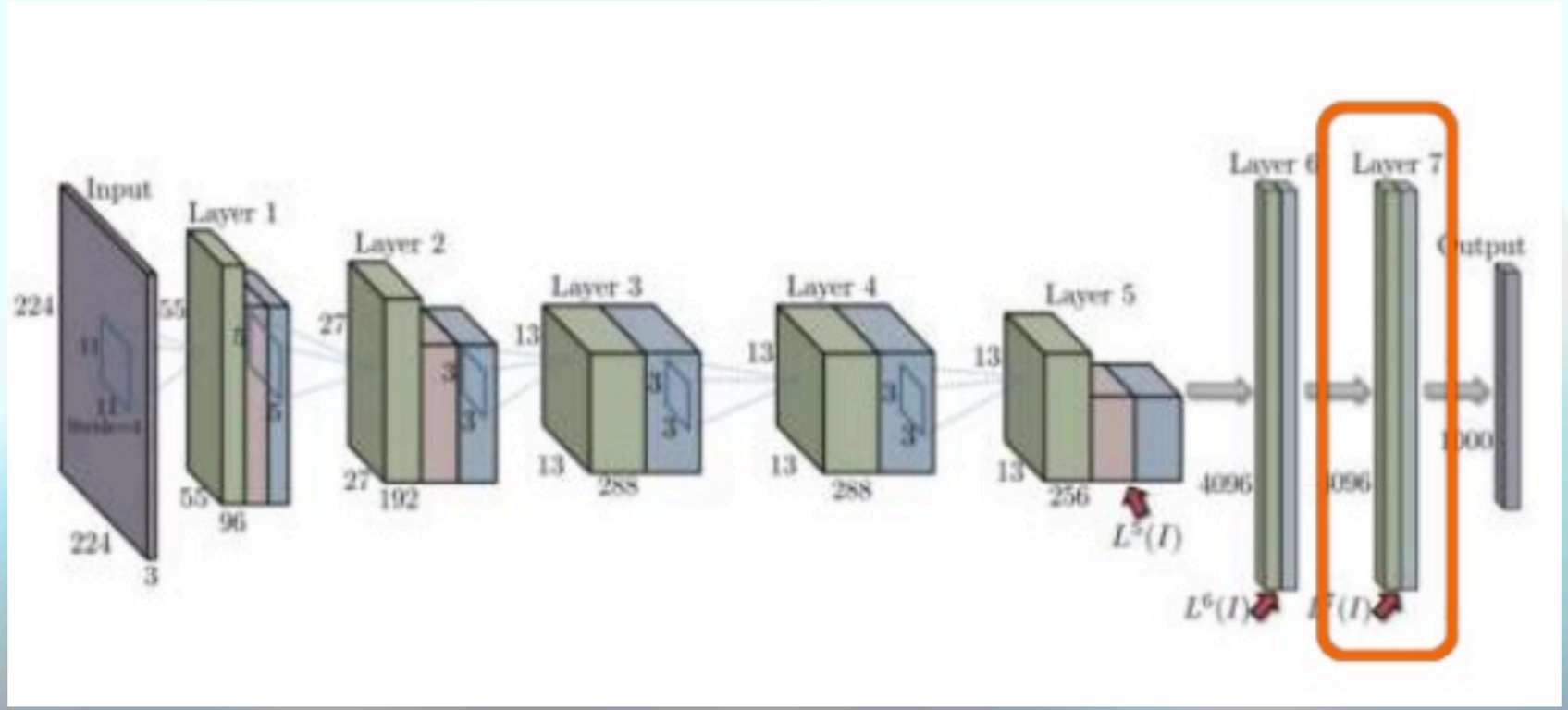
Como usar

- Escolha um modelo pre-treinado
- Remova as camadas de classificação (tendo em vista que a sua tarefa é outra)
- Adicione a nova camada de classificação para seu problema
- Fine-Tune
  - Você pode re-treinar as camadas finais usando seus dados.

Transfer Learning

Casos de uso - Extrator de características

- Usar uma rede pré-treinada somente como extrator de características
  - De posse dos vetores, qualquer algoritmo pode ser usado para classificação



Transfer Learning

Casos de Uso - Treinamento da Última Camada

- Manter toda a rede congelada e re-treinar somente a camada de saída, usando os exemplos da sua base de dados
- Ideal para pequenas bases de dados, nas quais não temos exemplos suficiente para treinar todas as camadas da rede.

Transfer Learning

Casos de Uso - Fine Tuning

- No caso anterior, toda a rede está congelada e somente a última camada é treinada.
- No Fine Tuning, algumas camadas são “descongeladas” e treinadas com os novos dados.
  - Nesse caso é necessário
    - 1- Treinar a camada final, como no passo anterior
    - 2-Descongelar as camadas
    - 3-Continuar o treinamento.

Google Colab

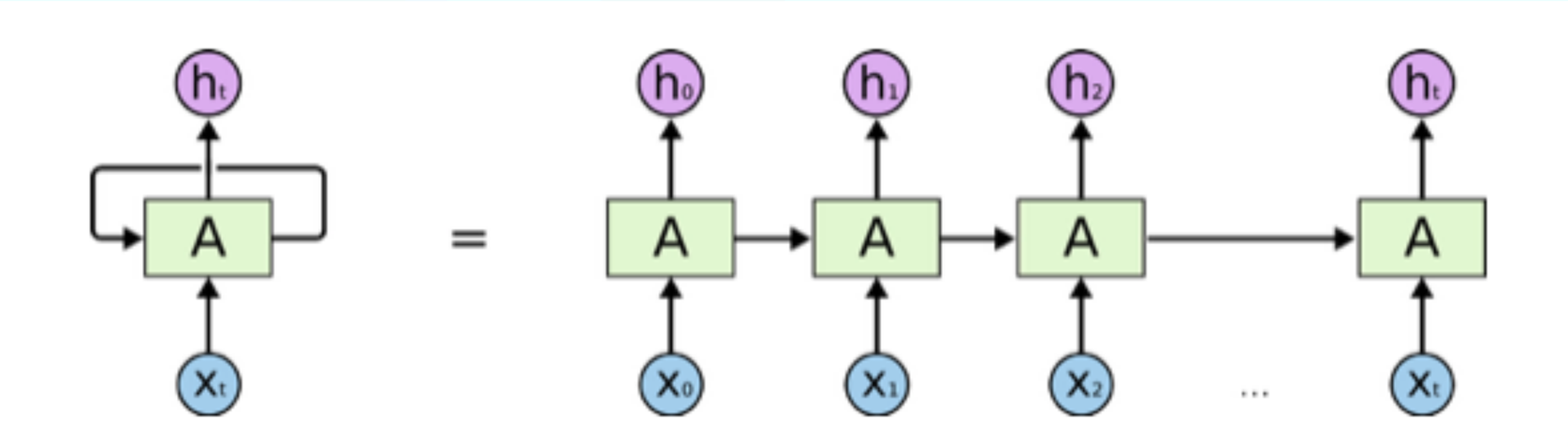
Recurrent Neural Networks

Redes Neurais Recorrentes

- Projetadas para reconhecer padrões com dependência sequencial (1982 - J. Hopfield, 1986 - M. Jordan).
- Processamento de Linguagem Natural
  - Voz
  - Genoma
  - Séries temporais
- Hipótese é que o ser humano não adquire o conhecimento do zero toda vez que aprende algo novo.
  - Ou seja, o conhecimento adquirido é persistente.

Redes Neurais Recorrentes

- Para que o conhecimento seja persistente, as RNNs (Recurrent Neural Nets) usam uma estrutura com laços
- Como pode-se observar, a camada escondida da rede recebe informação tanto da camada de entrada quanto de estados anteriores da camada escondida.



- A decisão tomada em **t-1** afeta a decisão que será tomada no tempo **t**
- Diferente das redes *feedforward*, a RNN tem uma memória.

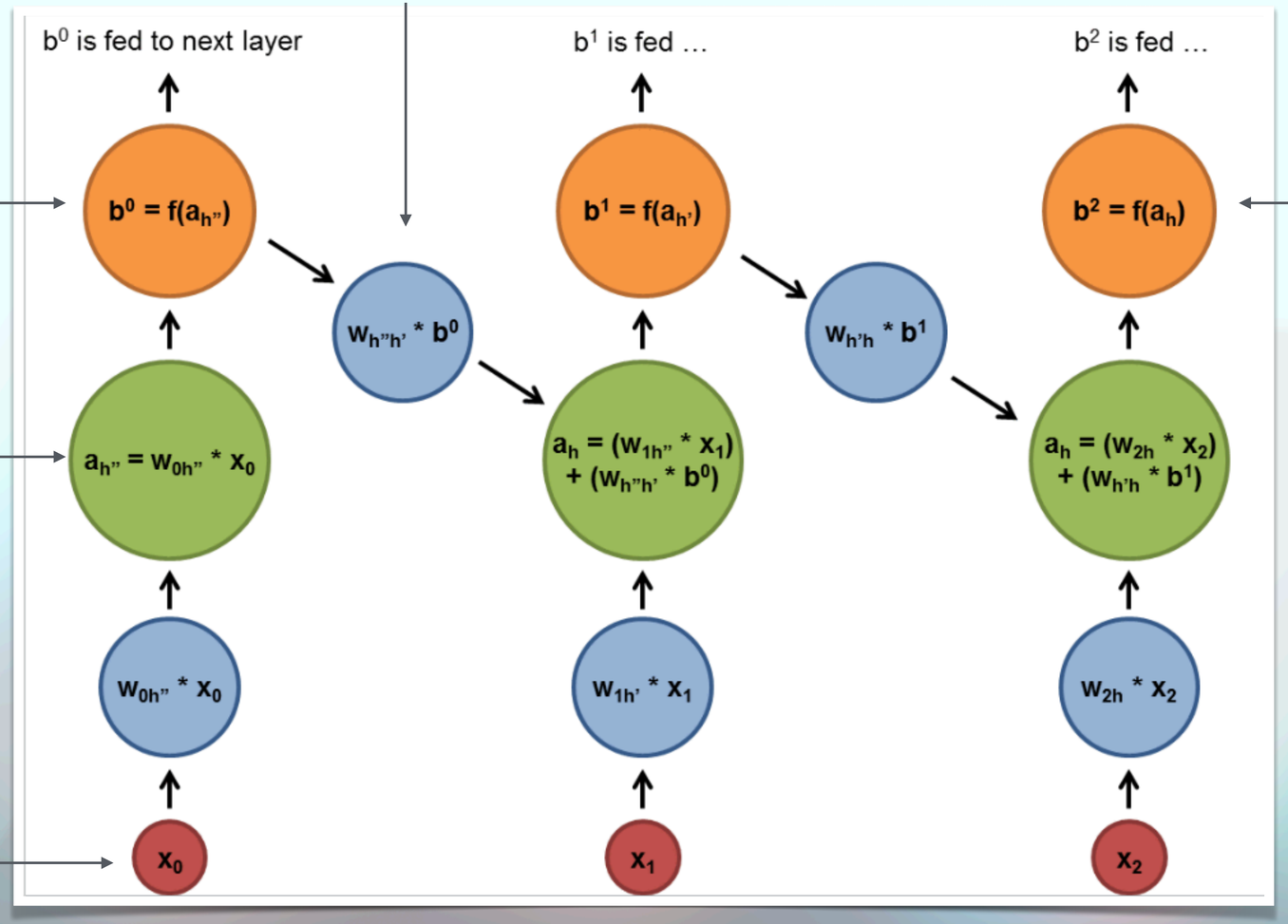
Redes Neurais Recorrentes

- A informação sequencial é preservada na camada escondida da RNN a qual difunde essa informação ao longo do tempo.
- Encontra correlações entre eventos separados ao longo do tempo.
- Essas correlações são conhecidas como **long-term dependencies**
- RNN compartilham pesos ao longo do tempo.

$$h_t = \phi(Wx_t + Uh_{t-1})$$

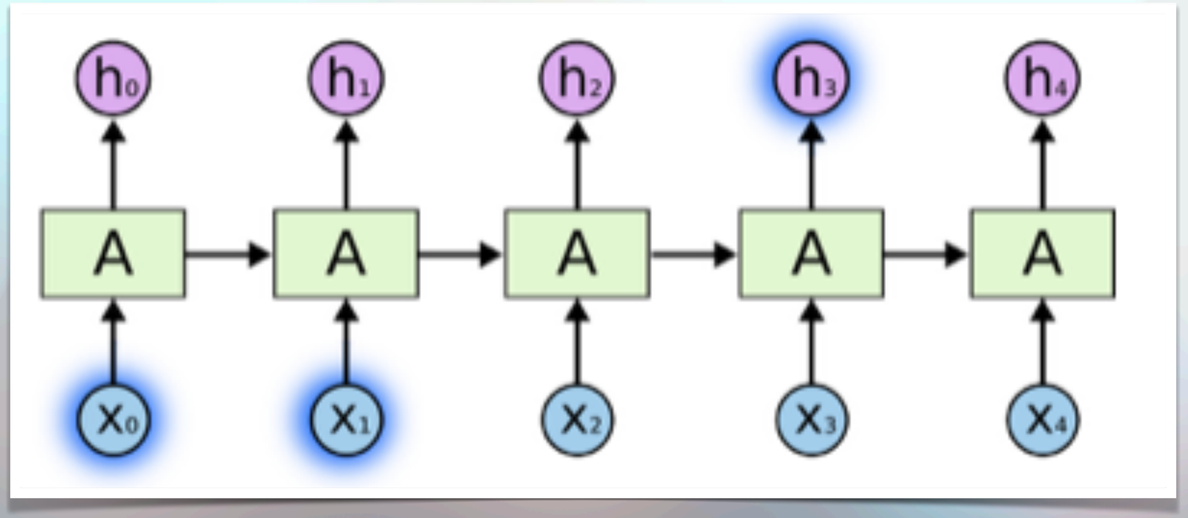
- h_t é a amada escondida no tempo t, W é a matriz de pesos, x_t é a entrada da rede, U é a matriz de transição utilizada para adicionar a informação contida em h_{t-1}

Redes Neurais Recorrentes



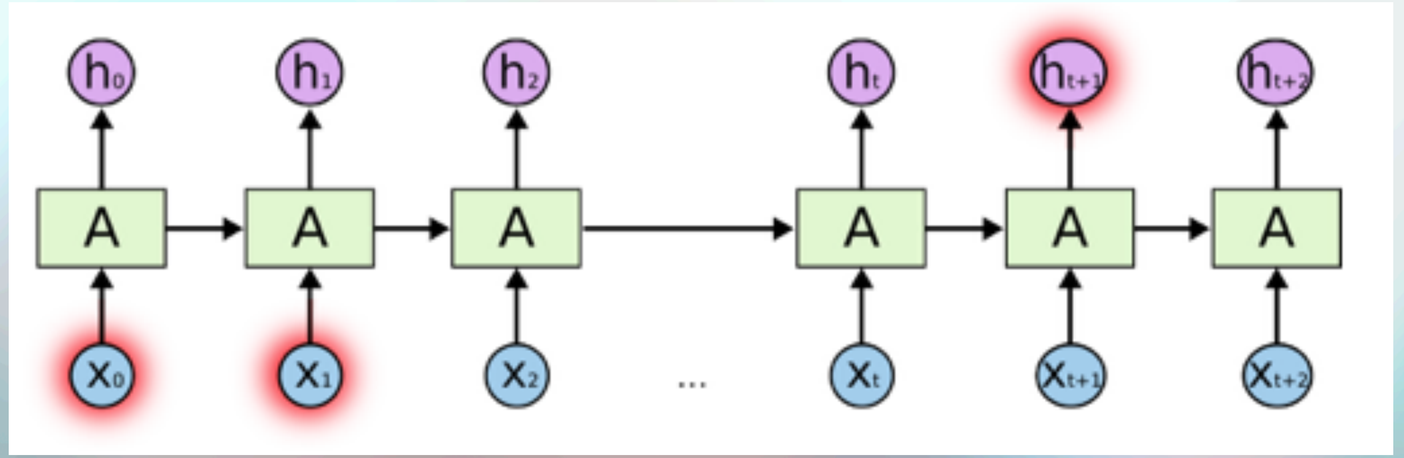
Redes Neurais Recorrentes

- O problema das dependências de longo termo (Long-Term Dependencies)
  - RNN tem um bom desempenho quando a distância entre a informação relevante e o lugar que ela é necessária é pequena.
  - Por exemplo, estimar a última palavra da sentença "as nuvens estão no céu''
  - Nesse caso o modelo não precisa de muito contexto para estimar a palavra correta.
  - Distância pequena para o contexto relevante (nuvens)



Redes Neurais Recorrentes

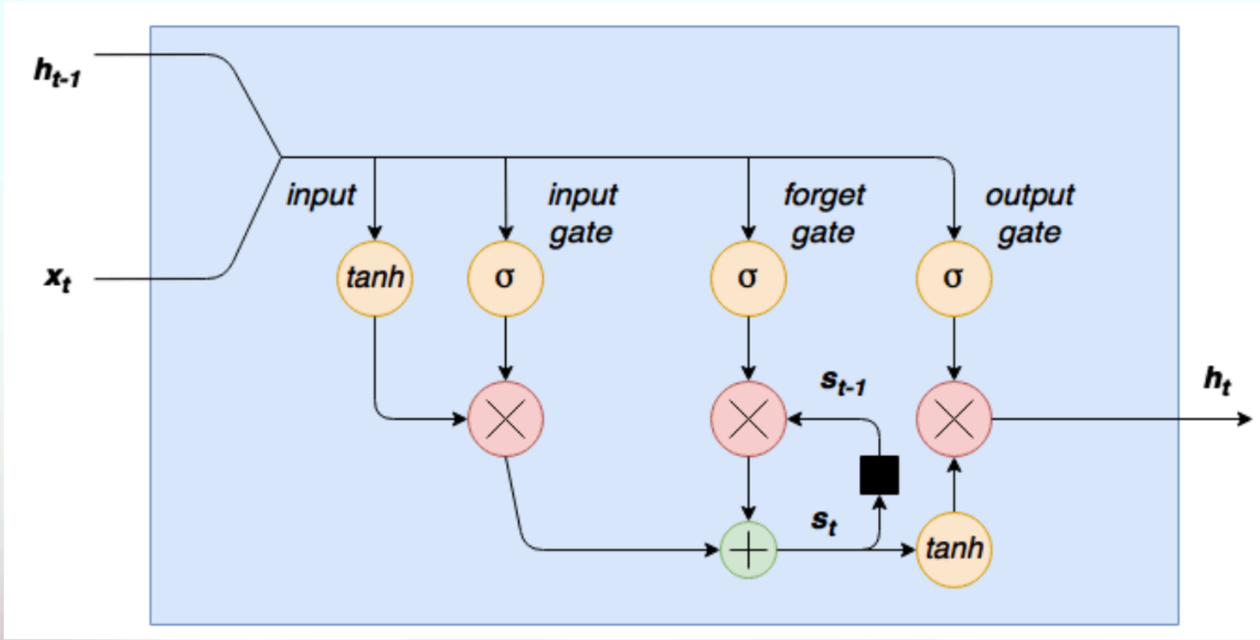
- O problema das dependências de longo termo (Long-Term Dependencies)
  - Mas na maioria dos casos, um contexto maior é necessário.
  - Eu cresci na França..... Eu falo fluentemente Francês"
  - Conforme a distância aumenta, a RNN não é capaz de fazer as conexões corretas (vanishing gradient problem)
  - Em teoria, a RNN é capaz de lidar com esse tipo de problema se os parâmetros da rede forem cuidadosamente selecionados, o que na prática é inviável.



Redes Neurais Recorrentes

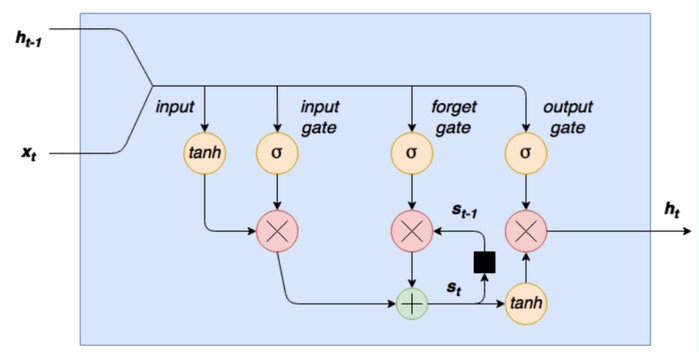
LSTM

- Projetada para mitigar os problemas das RNN:
- Neurônio LSTM (Long-Short Term Memory - Hochreiter et al, 1997)
- Estrutura interna mais complexa, que inclui "portões" (gates) que controlam o fluxo de informações.



LSTM

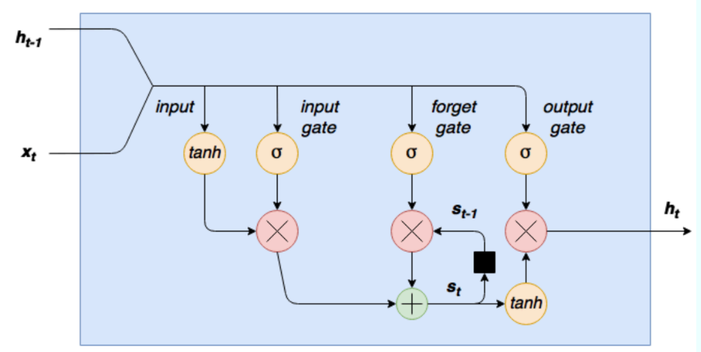
Input Gate



- Este portão tem duas partes principais que trabalham juntas (Tanh, Sigmoid):
- Sigmoid:
  - Decide quais novas informações da entrada atual (x_t) e do estado oculto anterior (h_{t-1}) são relevantes para serem **adicionadas** ao estado da célula. Atua como um "filtro" para o candidato a ser adicionado
- Tanh:
  - Cria um vetor de novos valores "candidatos" para serem adicionados ao estado da célula. A tangente hiperbólica produz valores entre -1 e 1, que podem representar a força e a direção da nova informação.
  - A saída da sigmoid (o "filtro de entrada") é multiplicada pelo vetor de candidatos da tanh. Isso decide quais partes do candidato são realmente "permitidas" a passar.

LSTM

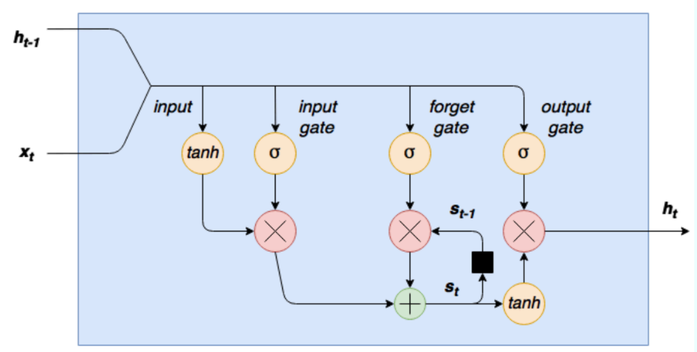
Forget Gate



- Decide quais informações do **estado da célula anterior (s_{t-1})** devem ser "esquecidas" ou descartadas.
- A sigmoide produz um valor entre 0 e 1 para cada elemento do estado da célula. Um valor próximo de 0 significa "esquecer completamente", enquanto um valor próximo de 1 significa "manter completamente".
- A saída deste portão (o "filtro de esquecimento") é multiplicada pelo s_{t-1} (o estado da célula do passo anterior).

LSTM

Atualização do Estado da Célula (+)

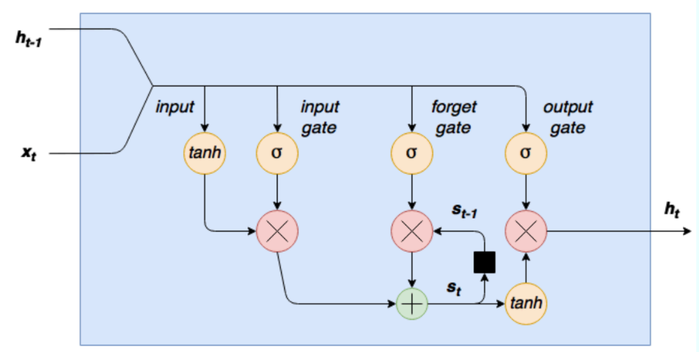


- O resultado do **Forget Gate** (informações de s_{t-1} que foram *mantidas*) é somado ao resultado do **Input Gate** (as *novas informações* que foram aprovadas).

- Isso gera o **novo estado da célula (s_t)**, que agora contém uma combinação seletiva do estado anterior e das novas informações relevantes. Este s_t é o que permite à LSTM manter informações por longos períodos.

LSTM

Output Gate



- Decide quais partes do **estado da célula recém-atualizado (s_t)** serão o estado oculto (h_t) para este passo de tempo. Este h_t é a saída que geralmente é usada para fazer previsões ou é passada para a próxima camada da rede.
  - A sigmoide produz um filtro baseado em x_t e h_{t-1}
  - O s_t (estado da célula) passa por uma função tanh.
  - A saída da sigmoide (o "filtro de saída") é multiplicada pelo resultado da tanh (s_t).
- Isso gera o **novo estado oculto (h_t)**, que é uma versão filtrada e processada do estado da célula, representando a informação relevante para a saída e para o próximo passo da sequência.

LSTM

Em resumo

- A LSTM funciona como um sistema de "bombeamento" de informações com válvulas controladas (os portões):
  - Forget Gate: Decide o que jogar fora da memória antiga.
  - Input Gate: Decide o que de novo é importante guardar na memória.
  - Estado da Célula (st): É a "memória principal" que flui através da rede, sendo seletivamente atualizada.
  - Output: Decide o que da memória principal é relevante para a saída atual e para ser o "estado oculto" passado adiante.
- Essa arquitetura complexa permite que as LSTMs capturem dependências de longo prazo de forma muito mais eficaz do que as RNNs tradicionais, o que as tornou uma peça chave para o avanço do Processamento de Linguagem Natural

LSTM

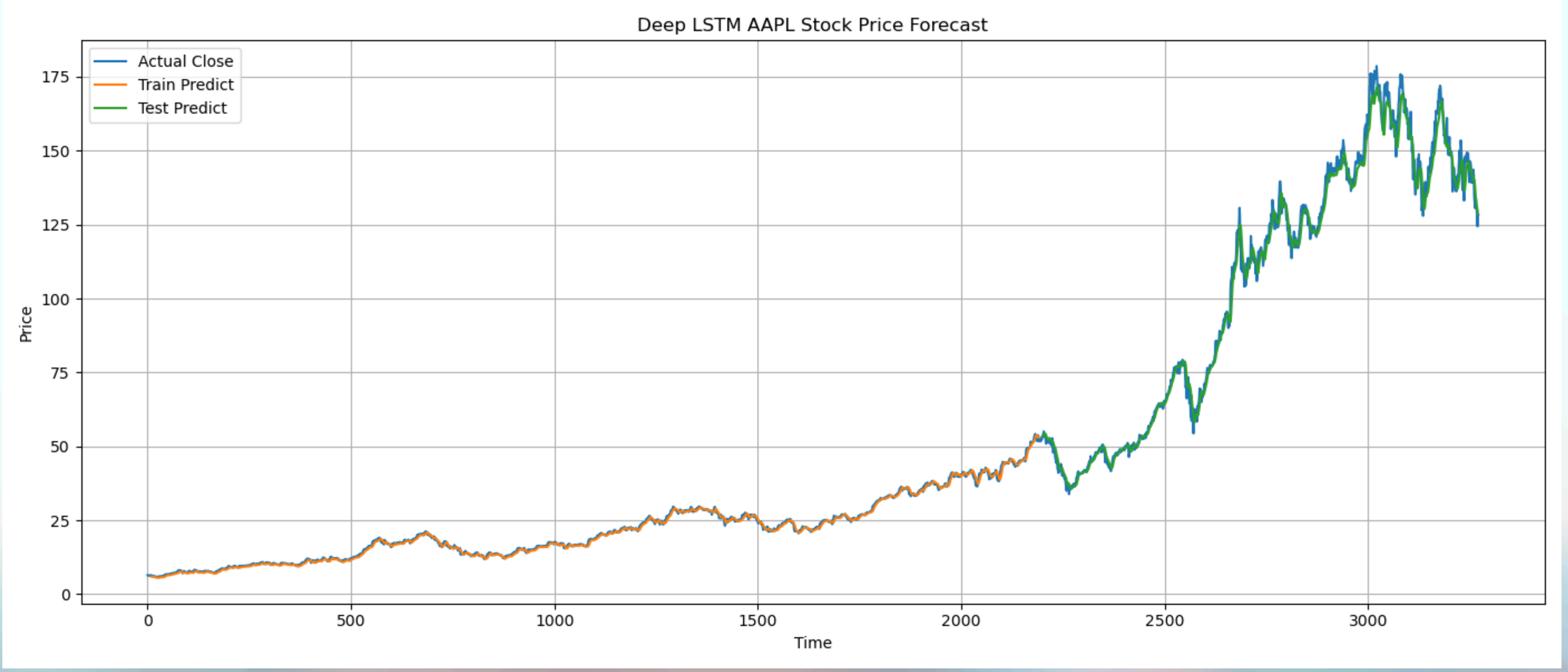
Exemplo

```python
# Deep LSTM
model = Sequential()

model.add(LSTM(64, return_sequences=True, input_shape=(look_back, 1)))
model.add(Dropout(0.2))
model.add(LSTM(32, return_sequences=False))
model.add(Dropout(0.2))
model.add(Dense(1))
model.compile(loss='mean_squared_error', optimizer='adam')

# Training
model.fit(trainX, trainY, epochs=50, batch_size=32, verbose=2)

# Predictions
trainPredict = model.predict(trainX)
testPredict = model.predict(testX)
```



LSTM

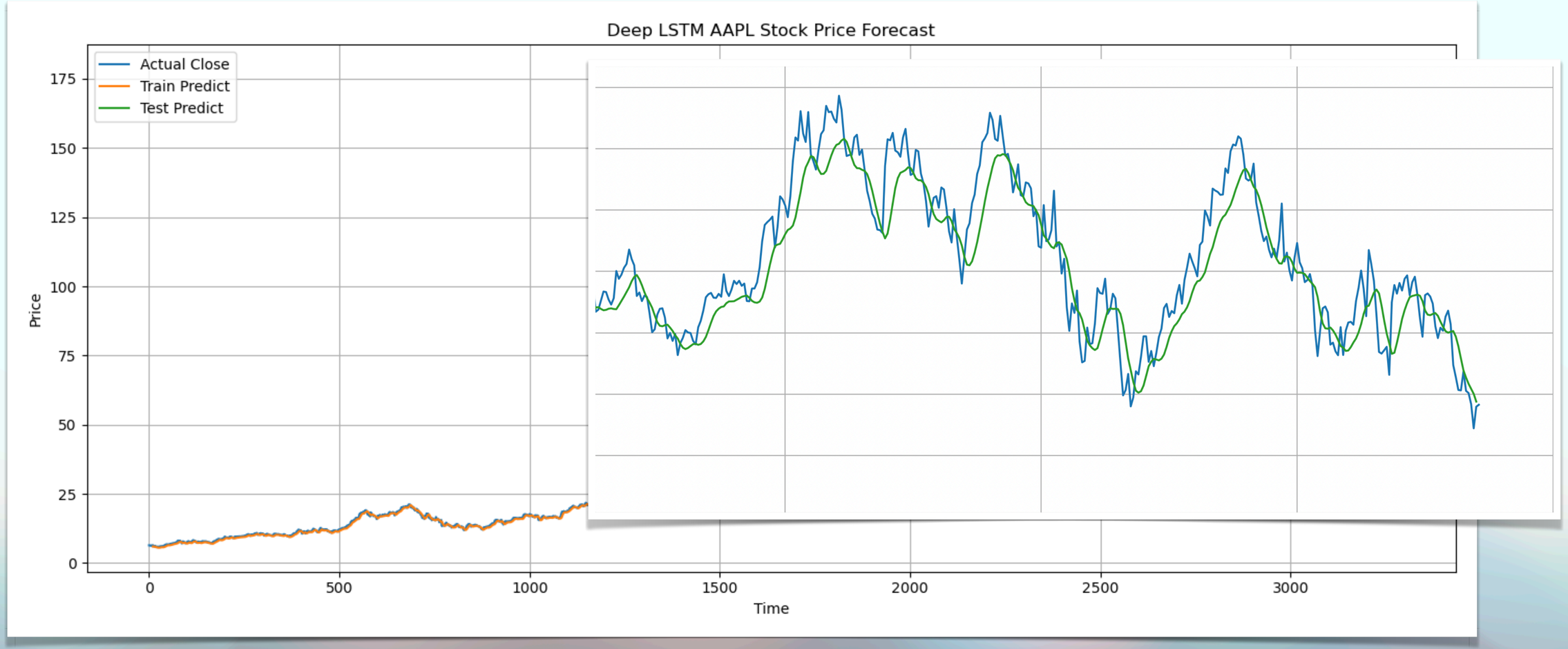
Exemplo

```python
# Deep LSTM
model = Sequential()

model.add(LSTM(64, return_sequences=True, input_shape=(look_back, 1)))
model.add(Dropout(0.2))
model.add(LSTM(32, return_sequences=False))
model.add(Dropout(0.2))
model.add(Dense(1))
model.compile(loss='mean_squared_error', optimizer='adam')

# Training
model.fit(trainX, trainY, epochs=50, batch_size=32, verbose=2)

# Predictions
trainPredict = model.predict(trainX)
testPredict = model.predict(testX)
```



LSTM

Limitações

- Apesar de lidarem melhor com dependências de longo prazo, as LSTMs (e RNNs em geral) ainda são fundamentalmente sequenciais.
- Essa dependência sequencial significa que as operações não podem ser facilmente paralelizadas em GPUs ou outros hardwares especializados.
  - Inviabiliza o treinamento com grandes bases
- Para sequências *extremamente* longas (centenas ou milhares de passos de tempo), a capacidade de reter informações relevantes do início da sequência ainda pode diminuir.
  - Depende muito da escolha dos parâmetros
- Como usar paralelismo em dados sequencias ?

Obrigado.